ulm.lam and the Field Guide

A Companion: Compiler Theory in a Linear-Algebra Compiler on MLIR (Multi-Level Intermediate Representation)

What the compiler does · how MLIR represents it · what the theory calls it

Describes ulm.lam at commit `4185542` (`41855425a8b9ea4ee279fd2ea4b31b45b53e1983`).
A companion to *Modern Compiler Concepts: A Field Guide.* Every example was run; commands, versions and outputs are in Appendix A.

How to Read This Companion

This companion connects three things that you already have: the compiler ulm.lam (Ulm Linear Algebra Mapping), your practical experience with MLIR, and the theory collected in *Modern Compiler Concepts: A Field Guide* (here: *the Field Guide*, cited as FG (Field Guide)). It does not repeat the Field Guide. It shows what ulm.lam does with one concrete program, how MLIR represents and checks it, and then names the concept and points to the Field Guide entry that explains it.

Every section follows the same order:

Input	the program or IR (intermediate representation) that goes in;
Representation	how MLIR holds it: types, attributes, operations, values;
Rule	the check, inference, rewrite or analysis that applies;
Output	what comes out (actual compiler output, not a sketch);
Theory	the theoretical name, with the exact Field Guide section.

Chapter 1 stands alone. It follows the GLS (generalized least squares) problem from the formula to the kernel calls and to numbers, and connects each step to its theory. The later chapters take up one question each and go deeper.

Where facts come from. Every property that the compiler uses has an origin. The text marks it with one of six tags:

declared	stated by the programmer (a type such as `Mat(:n, :n, :spd)`, or `assume`) and trusted, not proved
verified	checked structurally by an MLIR verifier when the IR is built or parsed (shapes, contradictions, result types)
inferred	derived by an inference rule from other facts ($X^{\mathsf{T}}M^{-1}X$ is SPD (symmetric positive definite) if M is SPD and X has full column rank)
analysis	established by an analysis of the program (which buffer a size comes from, whether an in-place write conflicts with a later read)
run time	checked when the generated code runs (array sizes, a failed Cholesky factorization)
not established	not checked anywhere; the result is wrong or unspecified if the fact is false

Boxes. A *Theory* box names the concept behind a mechanism and gives the Field Guide section. A *Not the same thing* box marks where an analogy stops, next to the example where it matters.

Links. Internal references are hyperlinks. A reference such as FG §1.7 *Refinement types* opens the Field Guide at that section when `modern-compiler-concepts.pdf` is in the same directory as this PDF (Portable Document Format) file (a link to its named destination `section.1.7`); otherwise the printed number and title locate it. Links to source code, such as `lib/UlmLamDialect.cpp:98`, go to the exact lines at commit `4185542` on GitHub. All section numbers and titles were checked against the current Field Guide source.

Code. Excerpts are copied from the sources at commit `4185542`; lines left out are marked `...`. IR listings are the actual output of the compiler, shortened only by omitting lines (also marked) and by wrapping long lines. Where a listing is schematic or an extension that does not exist in ulm.lam, it says so.

Notation. Matrices are real, X^{T} is the transpose, M^{-1} the inverse, $M^{-\mathsf{T}} = (M^{-1})^{\mathsf{T}}$. SPSD (symmetric positive semidefinite) complements SPD (symmetric positive definite), which appears in the tag table above. BLAS (Basic Linear Algebra Subprograms) and LAPACK (Linear Algebra PACKage) routine

names (`potrf`, `trsm`, …) are introduced where they first appear [13, 12]. Every abbreviation is spelled out in parentheses at its first use.

Contents

How to Read This Companion ii

1 A Complete Worked Example: Generalized Least Squares 1
- 1.1 The problem and its assumptions . . . 1
- 1.2 The Julia frontend code . . . 1
- 1.3 Tracing: from Julia to the first IR . . . 3
- 1.4 What the types encode, and what they do not . . . 4
- 1.5 Normal form . . . 5
- 1.6 Inferring properties . . . 6
- 1.7 Kernel selection . . . 7
- 1.8 From values to memory: bufferization, workspace and `info` . . . 9
- 1.9 The kernel sequence in C++ . . . 10
- 1.10 A numerical example . . . 11
- 1.11 The chapter in one table . . . 12

2 MLIR Provides the Infrastructure, ulm.lam Provides the Meaning 13
- 2.1 Who provides what . . . 13
- 2.2 A property set as a tensor encoding . . . 14
- 2.3 Operation verifiers: structural rules . . . 15
- 2.4 Result types must be justified . . . 16
- 2.5 Operation traits are not matrix properties . . . 16
- 2.6 What is present, and what is related by analogy . . . 17

3 Assumed, Verified, Inferred, Checked: Where Facts Come From 19
- 3.1 The facts and how they imply each other . . . 19
- 3.2 Declared facts: parameters and `assume` . . . 20
- 3.3 Inferred facts: the rules . . . 21
- 3.4 Facts established by analyses . . . 22
- 3.5 Facts checked at run time . . . 23
- 3.6 Three ways to know that a matrix is SPD . . . 23

4 Matrix Dimensions: Static, Named, Checked at Run Time 24
- 4.1 Static sizes: checked by the verifier . . . 24
- 4.2 Run-time sizes: anonymous in MLIR . . . 24
- 4.3 Named sizes: the frontend checks, MLIR records . . . 24
- 4.4 At the call: binding names to integers . . . 25

5 Values, Uses and Effects 27
- 5.1 One value, several uses . . . 27
- 5.2 Effects decide what may move . . . 28
- 5.3 Moving a factorization out of a loop . . . 28
- 5.4 Speculation and a false declaration . . . 29
- 5.5 Failure as a value, then as control flow . . . 30
- 5.6 Regions in MLIR and regions in memory management . . . 30

6 From Immutable Values to Mutable Storage **31**
6.1 Destination-passing style kernels . . . 31
6.2 Who may be overwritten . . . 32
6.3 Loop state in place . . . 32
6.4 One workspace for all temporaries . . . 32
6.5 Aliasing . . . 33
6.6 What each lowering keeps and drops . . . 33

7 From Algebra to Kernels **35**
7.1 Normalization: local rewrites to a fixpoint . . . 35
7.2 Kernel selection is a global optimization . . . 35
7.3 Lowering to C++: a dialect conversion . . . 37
7.4 Algebra over the reals and floating point . . . 37

8 Choosing an Evaluation Order **39**
8.1 The cost model . . . 39
8.2 Several plans and a run-time choice . . . 39
8.3 Decisions that depend on a trip count . . . 40
8.4 Specialization by element type, not by size . . . 41

9 Summary and Navigation **42**
9.1 Six answers . . . 42
9.2 Navigation table . . . 43

A Reproducing the Examples **46**
A.1 Commands . . . 46
A.2 Versions . . . 46
A.3 The project's own checks . . . 47
A.4 Files of this companion . . . 47

References **48**

CHAPTER 1

A Complete Worked Example: Generalized Least Squares

Answer. ulm.lam turns the formula $\beta = (X^\mathsf{T} M^{-1} X)^{-1} X^\mathsf{T} M^{-1} y$ into eleven calls: three copies, two Cholesky factorizations, a triangular solve with many right-hand sides, a symmetric rank-k update, a matrix-vector product and three triangular solves with one right-hand side. No inverse is formed, M is factorized once although it is inverted twice, and the matrix $L^{-1}X$ is computed once and used twice. The facts that make this possible (M is SPD, X has full rank, $X^\mathsf{T} M^{-1} X$ is SPD) are carried in MLIR tensor types; two of them are declared and one is inferred. The two factorizations are the only checks of these facts in the generated code.

This chapter follows one program through every representation. Figure 1.1 shows the stages; each section below explains one of them before the next one appears. All listings are the actual output of the commands in Appendix A (files in `outputs/gls/`).

1.1 The problem and its assumptions

In the linear model $y = X\beta + e$ with an error e of covariance $\sigma^2 M$, the GLS estimate of $\beta \in \mathbb{R}^m$ is

$$\beta = (X^\mathsf{T} M^{-1} X)^{-1} X^\mathsf{T} M^{-1} y, \qquad X \in \mathbb{R}^{n\times m},\ M \in \mathbb{R}^{n\times n},\ y \in \mathbb{R}^n.$$

The formula needs two facts:

1. M is SPD, so M^{-1} exists and is SPD;
2. X has full column rank (rank $X = m \le n$).

Together they make $S = X^\mathsf{T} M^{-1} X$ SPD: for $v \neq 0$, $Xv \neq 0$ by (2), and $v^\mathsf{T} S v = (Xv)^\mathsf{T} M^{-1} (Xv) > 0$ by (1). So S is invertible and β is well defined.

A numerical analyst does not form either inverse. With the Cholesky factorization $M = L_1 L_1^\mathsf{T}$ (L_1 lower triangular):

$$W = L_1^{-1} X, \quad z = L_1^{-1} y, \quad S = W^\mathsf{T} W = L_2 L_2^\mathsf{T}, \quad \beta = L_2^{-\mathsf{T}} L_2^{-1} W^\mathsf{T} z.$$

W appears twice; S is symmetric, so only one triangle is computed; S is SPD, so it is factorized by Cholesky as well. This is what ulm.lam derives, and the rest of the chapter shows how.

1.2 The Julia frontend code

The program, as in the README and in `examples/gls.jl`:

```
using UlmLam, LinearAlgebra

@lam function gls(X::Mat(:n, :m, :full_rank), M::Mat(:n, :n, :spd), y::Vec(:n))
    inv(X' * inv(M) * X) * X' * inv(M) * y
end

beta = gls(X, M, y)    # Julia arrays; the first call compiles
```

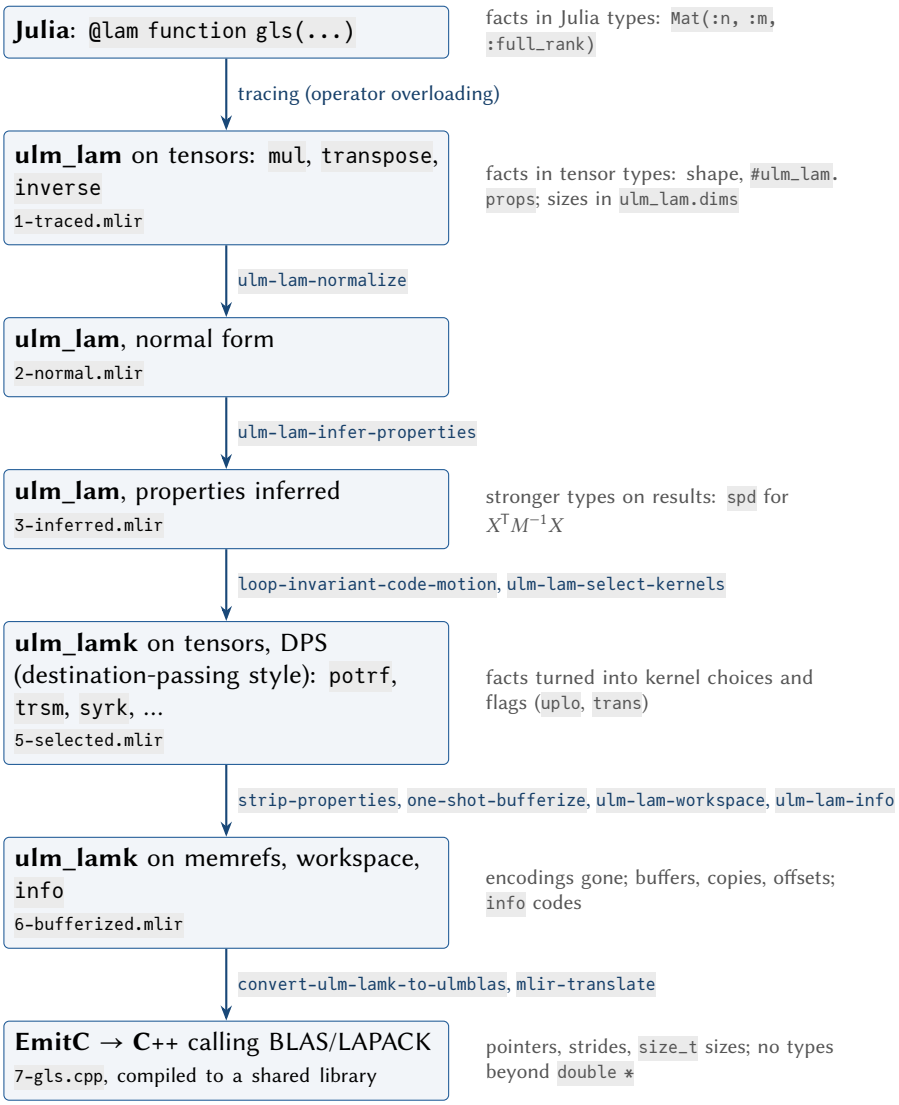


Figure 1.1: The stages of the GLS example. Left: the representation and the pass that produces it. Right: where the facts about the matrices live at that stage.

The body is the formula. The parameter types are not Julia types but ulm.lam types, values of the Julia structs `MatType` (`julia/UlmLam/src/frontend.jl:39`):

- `Mat(:n, :m, :full_rank)` is a matrix with n rows and m columns and the property `full_rank`. The symbols `:n` and `:m` are *run-time sizes*: their values are taken from the arrays at each call, and equal symbols mean equal sizes. A number instead of a symbol is a size fixed at compile time.
- `Mat(:n, :n, :spd)` is an $n \times n$ SPD matrix.
- `Vec(:n)` is a vector, represented as an $n \times 1$ matrix.

The macro `@lam` does not compile anything. It wraps the function so that the first call with Julia arrays traces the body, compiles the traced program into a shared library (about half a second), and calls it.

1.3 Tracing: from Julia to the first IR

Input	the Julia function, called once with placeholder values of type `LamValue`.
Representation	each Julia operation on a `LamValue` appends one line of MLIR text.
Rule	operator overloading: `*` emits `ulm_lam.mul`, `'` emits `ulm_lam.transpose`, `inv` emits `ulm_lam.inverse`.
Output	a `func.func` in the `ulm_lam` dialect (`1-traced.mlir`).
Theory	a tracing front end of an embedded DSL (domain-specific language), FG §3.7 *Embedded DSLs and tracing/AST-capturing front ends* (AST: abstract syntax tree).

The overloaded product (`julia/UlmLam/src/frontend.jl:222`) already checks the inner sizes, by comparing sizes that may be symbols:

```
function Base.:*(a::LamValue, b::LamValue)
    ...
    ta, tb = mat(a, "*"), mat(b, "*")
    ta.cols == tb.rows ||
        throw(LamError("product of $ta and $tb: $(ta.cols) columns but " *
                       "$(tb.rows) rows"))
    op!("ulm_lam.mul", [a, b], newtype(ta.rows, tb.cols))
end
```

The signature. The traced function begins (one line in the file, wrapped here):

```
func.func @gls(%s.n: index, %s.m: index,
    %p.X: tensor<?x?xf64, #ulm_lam.props<full_rank>> {ulm_lam.in, ulm_lam.dims = [1, 0]},
    %p.M: tensor<?x?xf64, #ulm_lam.props<spd>> {ulm_lam.in, ulm_lam.dims = [0, 0]},
    %p.y: tensor<?x1xf64> {ulm_lam.in, ulm_lam.dims = [0]})
    -> (tensor<?x1xf64> {ulm_lam.out, ulm_lam.dims = [1]}) {
```

Read it from left to right:

- `%s.n` and `%s.m` are the run-time sizes, as `index` parameters of the function. The generated C function receives them as `size_t` arguments.
- `%p.X` has type `tensor<?x?xf64, #ulm_lam.props<full_rank>>`: a rank-2 tensor of `f64` with two dynamic dimensions (`?`) and, in the *encoding* slot of the builtin tensor type, the property set `full_rank`.
- `ulm_lam.dims = [1, 0]` is an argument attribute. It says which `index` parameter each dynamic dimension is: the rows of `%p.X` are parameter 1 (m), its columns parameter 0 (n). So `%p.X` is $m \times n$, not $n \times m$: Julia stores matrices column-major, the ulm.lam ABI (application binary interface) is row-major, and the memory of the Julia matrix X *is* the row-major matrix X^{T}. The frontend types the parameter as X^{T} (`abitype`, `julia/UlmLam/src/frontend.jl:100`) and hands the traced function `transpose(%p.X)`, which is X. No data is copied for this.
- `ulm_lam.in` marks a read-only argument, `ulm_lam.out` the result, which the caller allocates.

The body. Twelve operations, one per Julia operation:

```
%v1 = ulm_lam.transpose %p.X  : ... -> tensor<?x?xf64>        // X   (Julia's X)
%v2 = ulm_lam.transpose %p.M  : ... -> tensor<?x?xf64>        // M
%v3 = ulm_lam.transpose %v1   : ...                           // X'
%v4 = ulm_lam.inverse %v2     : ...                           // inv(M)
%v5 = ulm_lam.mul %v3, %v4    : ...                           // X' * inv(M)
%v6 = ulm_lam.mul %v5, %v1    : ...                           // ... * X
%v7 = ulm_lam.inverse %v6     : ...                           // inv(...)
%v8 = ulm_lam.transpose %v1   : ...                           // X'
%v9 = ulm_lam.inverse %v2     : ...                           // inv(M), again
%v10 = ulm_lam.mul %v7, %v8   : ...
```

Information	Where in the IR	Origin
element type `f64`	tensor element type	declared by the call (Float64 arrays; Float32 arrays compile an `f32` version)
rank 2	tensor rank; ODS (Operation Definition Specification) constraint `2DTensorOf<[F32, F64]>`	verified by the generated operand checks
static sizes	tensor shape	verified by op verifiers when static
which run-time size	`index` parameters and the argument attribute `ulm_lam.dims`	declared by the frontend; read by an analysis (`dimOf`, Section 4.3)
matrix properties	tensor encoding `#ulm_lam.props<...>`	declared on parameters; inferred on results (Section 1.6)
may the function overwrite it?	argument attributes `ulm_lam.in` / `ulm_lam.inout`	declared; drives bufferization (Section 1.8)
the entries	nowhere (run-time values)	checked only indirectly, by the factorizations run time

Table 1.1: Where the GLS program's information lives in the first IR.

```
%v11 = ulm_lam.mul %v10, %v9  : ...
%v12 = ulm_lam.mul %v11, %p.y : ... -> tensor<?x1xf64>
return %v12 : tensor<?x1xf64>
```

(The comments are added here; the types after `:` are shortened.) Every intermediate result has the plain type `tensor<?x?xf64>`: the frontend writes no properties for results. It also records exactly what the Julia code does: Julia's left-to-right `*` gives binary products nested to the left, and `inv(M)` appears twice (`%v4`, `%v9`) because the formula contains it twice.

> **Theory (FG §3.7).** ulm.lam is an embedded DSL with a *tracing* front end: the host language runs the program on symbolic stand-ins and records the operations, as JAX does with Python. Host-language control flow is evaluated during tracing and disappears; only the recorded operations reach the compiler. What is known when (Julia values at trace time, sizes at call time, entries at run time) is a question of binding times, the sixth thread of the Field Guide's conceptual map, and of FG §8.6 *Multi-stage programming and binding-time analysis.*

1.4 What the types encode, and what they do not

Before the first pass runs, the IR carries everything the compiler will know about the problem. Table 1.1 lists where each piece of information is.

Why the properties are a tensor encoding. The builtin `RankedTensorType` has an optional *encoding* attribute. ulm.lam defines an attribute, `#ulm_lam.props<...>`, and places it there, instead of defining its own matrix type. Upstream passes and One-Shot Bufferize then see ordinary tensors, and the properties are removed in one step (`ulm-lam-strip-properties`) once they are no longer needed. The attribute is a bit set over fifteen named facts; its declaration is in `include/ulm_lam/UlmLamAttrs.td:1` and is discussed in Section 2.2.

What the type of `%p.M` says. `tensor<?x?xf64, #ulm_lam.props<spd>>` reads "some $n \times n$ matrix that is SPD". The compiler takes this as given. Nothing in the IR proves that the caller's array is symmetric or positive definite, and the compiler never reads the entries. This is the first boundary of the system:

declared facts enter at the parameters (and at `ulm_lam.assume`), and everything else is derived from them.

> **Theory (FG §1.7).** A type that restricts a base type by a predicate, $\{A : \mathbb{R}^{n\times n} \mid A \text{ is SPD}\}$, is a *refinement type*, FG §1.7 *Refinement types.* ulm.lam's predicates come from a fixed set of fifteen atoms and their conjunctions, like the qualifiers of liquid types [6, 5]. Chapters 2 and 3 make the comparison precise, including what is missing (no proof obligations at call sites, no predicates that relate a matrix to other values).

1.5 Normal form

Input	the traced IR of Section 1.3.
Representation	the same ops; patterns are C++ classes derived from `OpRewritePattern`.
Rule	fold $A^{\mathsf{TT}} \to A$; fold $A^{\mathsf{T}} \to A$ when A is symmetric; flatten nested products into one n-ary `ulm_lam.mul` (if the inner product has no other use).
Output	`2-normal.mlir`, below.
Theory	greedy pattern rewriting to a fixpoint, FG §3.4 *Pattern rewriting: greedy drivers, canonicalization, PDL* (PDL: Pattern Descriptor Language); termination, FG §3.5 *Term rewriting theory: confluence, termination, strategies.*

The pass `ulm-lam-normalize` collects the canonicalization patterns of the ops and runs MLIR's greedy driver (`applyPatternsGreedily`, `lib/UlmLamPasses.cpp:105`). The output (types shortened):

```
%0 = ulm_lam.transpose %arg2 : <X^T, full_rank> -> tensor<?x?xf64>          // X
%1 = ulm_lam.inverse %arg3 : <M, spd> -> tensor<?x?xf64>                    // M^-1
%2 = ulm_lam.mul %arg2, %1, %0 : ... -> tensor<?x?xf64>                     // X^T M^-1 X
%3 = ulm_lam.inverse %2 : tensor<?x?xf64> -> tensor<?x?xf64>
%4 = ulm_lam.inverse %arg3 : <M, spd> -> tensor<?x?xf64>                    // M^-1
%5 = ulm_lam.mul %3, %arg2, %4, %arg4 : ... -> tensor<?x1xf64>              // S^-1 X^T M^-1 y
return %5 : tensor<?x1xf64>
```

Three rules did the work:

- `FoldTranspose` (`lib/UlmLamDialect.cpp:604`) replaced M^{T} by M. It may do so only because the *type* of `%p.M` says `spd`, which implies `symmetric`. A rewrite that depends on a property in a type: this is the first place where a declared fact changes the program.
- The same pattern folded $(X^{\mathsf{T}})^{\mathsf{T}}$ to the parameter itself, so X^{T} is now `%arg2` and X is `%0`.
- `Flatten` (`lib/UlmLamDialect.cpp:578`) merged the left-nested binary products into one three-factor and one four-factor `ulm_lam.mul`. An n-ary product leaves the parenthesization open; choosing it is the job of kernel selection.

Note what did *not* happen: `%1` and `%4` are still two `inverse` ops of the same value. The normalization pipeline contains no CSE (common subexpression elimination). The factorization will be shared anyway, by a memo table in kernel selection (Section 1.7).

> **Theory (FG §3.4, FG §3.5).** Each pattern is a rewrite rule $\ell \Rightarrow r$ with a side condition ("A is symmetric", "the inner product has one use"). The greedy driver applies rules until none matches, a fixpoint of the rewrite relation. That the fixpoint is reached is a termination argument (each rule decreases a well-founded measure, as the pass description in `include/ulm_lam/UlmLamPasses.td:8` states); whether it is unique is confluence. Both are questions of FG §3.5 *Term rewriting theory: confluence, termination, strategies.*

1.6 Inferring properties

Input	the normal form.
Representation	result types without properties (`tensor<?x?xf64>`).
Rule	for every op in program order, compute the result type from the operand types (`InferTypeOpInterface::inferReturnTypes`) and replace the old one.
Output	`3-inferred.mlir`: M^{-1} is SPD, $X^{\mathsf{T}}M^{-1}X$ is SPD.
Theory	type inference in synthesis mode, FG §1.10 *Type inference beyond Hindley–Milner: bidirectional and constraint-based*; the property set as an abstract domain, FG §5.2 *Abstract interpretation: Galois connections, widening, narrowing.*

The pass `ulm-lam-infer-properties` (`lib/UlmLamPasses.cpp:139`) changes only types. The interesting lines of its output:

```
%1 = ulm_lam.inverse %arg3 : tensor<?x?xf64, #ulm_lam.props<spd>>
       -> tensor<?x?xf64, #ulm_lam.props<square, symmetric, spd, non_singular, full_rank>>
%2 = ulm_lam.mul %arg2, %1, %0 : ... -> tensor<?x?xf64, #ulm_lam.props<spd>>
%3 = ulm_lam.inverse %2 : tensor<?x?xf64, #ulm_lam.props<spd>>
       -> tensor<?x?xf64, #ulm_lam.props<square, symmetric, spd, non_singular, full_rank>>
```

M^{-1} is SPD. The rule for `inverse` keeps the properties that survive inversion (`inverseProperties`, a mask). The list printed for `%1` is the *closure* of `spd`: SPD implies symmetric, square and non-singular, and square plus non-singular implies full rank (`closure`, `lib/UlmLamDialect.cpp:97`).

$X^{\mathsf{T}}M^{-1}X$ is SPD. The rule for `mul` recognizes products that are their own transpose. Its core (`lib/UlmLamDialect.cpp:238`):

```
/// Products that read the same backwards when transposed,
/// F_1 ... F_k with F_i^T = F_(k+1-i):
///   Z^T Z      (k even)           spsd, spd if Z has full column rank;
///   Z^T M Z    (k odd, M middle)  symmetric if M is, spsd if M is, spd if
///                                 M is spd and Z has full column rank.
P palindromeProperties(ValueRange f, ArrayRef<P> ps)
{
    size_t k = f.size(), h = k / 2;
    ...
    for (size_t i = 0; i < h; ++i) {
        auto [a, ta] = leafOf(f[i]);          // (matrix, transposed?)
        auto [b, tb] = leafOf(f[k - 1 - i]);
        if (a != b || (ta == tb && !isSymmetric(a)))
            return P::general;
    }
    ...
    P m = ps[h];                              // the middle factor
    if (has(m, P::spd) && fullColumnRank(z, pz))
        return P::spd;
    ...
```

For `mul %arg2, %1, %0`: the outer factors are `%arg2` (X^{T}, the parameter) and `%0` (X, a transpose of `%arg2`); `leafOf` reduces both to `%arg2` with opposite transposition, so the product has the form $Z^{\mathsf{T}}MZ$. The middle factor `%1` is SPD. `fullColumnRank` accepts $Z = X$ because X is `full_rank` *and its sizes are dynamic*: the source comment (`lib/UlmLamDialect.cpp:222`) says that a full-rank matrix of unknown shape is taken to have full column rank, since the formula inverts $X^{\mathsf{T}}M^{-1}X$ and needs it. Hence `spd`.

Where each fact came from.

M is SPD	declared (`Mat(:n, :n, :spd)`)
X has full rank	declared (`Mat(:n, :m, :full_rank)`)
$n \geq m$, i.e. full *column* rank	not established at compile time: taken for granted because the sizes are dynamic; run time indirectly, if S is singular its Cholesky factorization fails
M^{-1} is SPD	inferred from M SPD
$X^{\mathsf{T}}M^{-1}X$ is SPD	inferred from the three facts above, by the palindrome rule

The inferred fact is exactly as reliable as the facts it was inferred from. It is a theorem about real matrices, *conditional on* the declarations.

> **Theory (FG §1.10, FG §5.1, FG §5.2).** Computing result types bottom-up from operand types is type *synthesis* in the sense of bidirectional typing, FG §1.10 *Type inference beyond Hindley–Milner: bidirectional and constraint-based.* There are no type variables and no unification: one walk in program order suffices because SSA (static single assignment) values are defined before they are used and the IR here has no cycles. The property sets with "absence means unknown" form a lattice ordered by implication, and each op's rule is a sound transfer function on it: FG §5.1 *Lattices, monotone frameworks and fixpoints (a sharper recap)* and FG §5.2 *Abstract interpretation: Galois connections, widening, narrowing.* Section 3.1 develops both views.

1.7 Kernel selection

Input	the inferred IR: two products with inverses, with properties on every value.
Representation	each product becomes a sequence of *leaves*: matrices (possibly transposed) and triangular solves.
Rule	factorize each inverted matrix once, by its properties; parenthesize each product by a dynamic program over kernels and flop (floating-point operation) counts; reuse a sub-product computed earlier, also transposed.
Output	`ulm_lamk` kernels on tensors (`5-selected.mlir`).
Theory	instruction selection by tiling with dynamic programming, FG §4.5 *Instruction selection: tree/DAG tiling, SelectionDAG vs GlobalISel* (DAG: directed acyclic graph); progressive lowering, FG §3.3 *Progressive lowering and the dialect conversion framework.*

The pass `ulm-lam-select-kernels` (class `Selector`, `lib/UlmLamPasses.cpp:318`) does three things for GLS.

1. Factorizations from properties. Every inverted matrix is factorized once, directly after its definition (`factorise`, `lib/UlmLamPasses.cpp:855`). The choice is read off the type: triangular: no factorization; `spd`: Cholesky (`potrf`, LAPACK's Cholesky factorization of a positive definite matrix); otherwise: LU (lower–upper) factorization with partial pivoting (`getrf`). Results are memoized per matrix, so `%1` and `%4`, both `inverse %arg3`, share one `potrf`. An inverse becomes solves: $M^{-1} = L_1^{-\mathsf{T}}L_1^{-1}$.

2. The first product. $X^{\mathsf{T}}M^{-1}X$ becomes the leaves X^{T}, $L_1^{-\mathsf{T}}$, L_1^{-1}, X. A dynamic program over intervals of leaves (`solveChain`, `lib/UlmLamPasses.cpp:1059`) considers, for each interval, a `gemm` of two sub-intervals, a solve applied from the left or right in place, and the *Gram* form $Z^{\mathsf{T}}Z$ when the left half is the transpose of the right half. Here the halves are $X^{\mathsf{T}}L_1^{-\mathsf{T}}$ and $L_1^{-1}X$, transposes of each other, so the product is $W^{\mathsf{T}}W$ with $W = L_1^{-1}X$: one `trsm` (triangular solve with many right-hand sides, BLAS level 3, n^2m flops) and one `syrk` (symmetric rank-k update, one triangle, m^2n flops). The result S is SPD by its type, so it is factorized by `potrf`.

3. The second product. $S^{-1}X^{\mathsf{T}}M^{-1}y$ becomes $L_2^{-\mathsf{T}}$, L_2^{-1}, X^{T}, $L_1^{-\mathsf{T}}$, L_1^{-1}, y. The interval $X^{\mathsf{T}}L_1^{-\mathsf{T}}$ is the transpose of W, which the first product computed; the table of computed products is keyed by leaves and transposition, so it costs nothing and W is reused. What remains is $z = L_1^{-1}y$ (`trsv`, triangular solve with one right-hand side), $W^{\mathsf{T}}z$ (`gemv`, matrix-vector product), and two `trsv` with L_2.

Output. After `ulm-lam-strip-properties` and `canonicalize` (`5-selected.mlir`; constants omitted):

```
%result, %info = ulm_lamk.potrf uplo(lower) outs(%arg3 : tensor<?x?xf64>) -> tensor<?x?xf64>
%0 = tensor.empty(%arg1, %arg1) : tensor<?x?xf64>
%1 = tensor.empty(%arg0, %arg1) : tensor<?x?xf64>
%2 = ulm_lamk.gecopy trans(trans) ins(%arg2 : tensor<?x?xf64>) outs(%1 : tensor<?x?xf64>) -> tensor<?x?
   ↪ xf64>
%3 = ulm_lamk.trsm side(left) uplo(lower) trans(none) diag(non_unit) alpha(%cst_0)
       ins(%result : tensor<?x?xf64>) outs(%2 : tensor<?x?xf64>) -> tensor<?x?xf64>
%4 = ulm_lamk.syrk uplo(lower) trans(trans) alpha(%cst_0) beta(%cst)
       ins(%3 : tensor<?x?xf64>) outs(%0 : tensor<?x?xf64>) -> tensor<?x?xf64>
%result_1, %info_2 = ulm_lamk.potrf uplo(lower) outs(%4 : tensor<?x?xf64>) -> tensor<?x?xf64>
%5 = ulm_lamk.trsv uplo(lower) trans(none) diag(non_unit)
       ins(%result : tensor<?x?xf64>) outs(%arg4 : tensor<?x1xf64>) -> tensor<?x1xf64>
%6 = tensor.empty(%arg1) : tensor<?x1xf64>
%7 = ulm_lamk.gemv trans(trans) alpha(%cst_0) beta(%cst)
       ins(%3, %5 : tensor<?x?xf64>, tensor<?x1xf64>) outs(%6 : tensor<?x1xf64>) -> tensor<?x1xf64>
%8 = ulm_lamk.trsv uplo(lower) trans(none) diag(non_unit)
       ins(%result_1 : tensor<?x?xf64>) outs(%7 : tensor<?x1xf64>) -> tensor<?x1xf64>
%9 = ulm_lamk.trsv uplo(lower) trans(trans) diag(non_unit)
       ins(%result_1 : tensor<?x?xf64>) outs(%8 : tensor<?x1xf64>) -> tensor<?x1xf64>
return %9 : tensor<?x1xf64>
```

Each kernel is in destination-passing style (DPS) [10]: the operand that BLAS overwrites is the `outs` operand, and on tensors the op returns the new value of it. `potrf` "overwrites" `%arg3`, which is the caller's M; on tensors that is harmless (a new value is returned), and bufferization will decide what it means for memory. `%3`, the matrix W, has two uses (`syrk` and `gemv`): one value, used twice. `gecopy trans(trans)` makes X (the transpose of `%arg2`) into a fresh row-major matrix that `trsm` can overwrite.

Julia's `@code_lam` prints the same sequence in the user's terms:

```
gls(X::n×m full_rank, M::n×n spd, y::n)
  potrf    L1 ← chol(M)                    M = L1 L1ᵀ, Cholesky
  copy     T1 ← X
  trsm     T1 ← L1⁻¹ T1
  syrk     T2 ← T1ᵀ T1                     one triangle only
  potrf    L2 ← chol(T2)                   T2 = L2 L2ᵀ, Cholesky
  trsv     T3 ← L1⁻¹ y
  gemv     T4 ← T1ᵀ T3
  trsv     T4 ← L2⁻¹ T4
  trsv     T4 ← L2⁻ᵀ T4
  return T4
if a factorisation fails, the call throws InfoError with
  info 1:  Cholesky factorisation of M: not positive definite
  info 2:  Cholesky factorisation of a computed matrix: not positive definite
```

> **Theory (FG §4.5, FG §3.3).** Kernel selection is instruction selection with BLAS and LAPACK as the instruction set: leaves are operands, kernels are tiles that cover an interval of the product, and the dynamic program picks the cheapest tiling, as in tree tiling with dynamic programming, FG §4.5 *Instruction selection: tree/DAG tiling, SelectionDAG vs GlobalISel.* The interval structure is that of the matrix chain problem (generalized to solves and transposes by Barthels et al. [3]). The pass replaces one dialect by another in a single step and keeps the structure needed later (DPS, flags), which is *progressive lowering*, FG §3.3 *Progressive lowering and the dialect conversion*

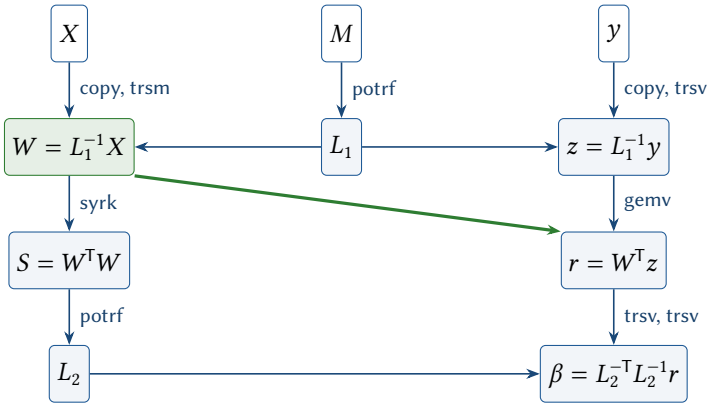


Figure 1.2: Values and dependences of the selected kernels. W (green) has two uses; L_1 has two uses. Each value is computed once.

framework. It does *not* use MLIR's dialect conversion framework: it is a custom C++ traversal (Section 7.2).

Not the same thing. The selected sequence equals the formula *over the real numbers*. In floating point it is a different computation with different rounding: W^TW is not bitwise $(X^TM^{-1})X$, and its accuracy depends on the condition number of W squared. Section 1.10 shows a difference in the last bit; Section 7.4 shows a case where the two differ completely.

1.8 From values to memory: bufferization, workspace and `info`

Input	kernels on tensors in DPS; read-only arguments marked `ulm_lam.in`.
Representation	after bufferization: kernels on `memrefs`, with no results.
Rule	One-Shot Bufferize writes a kernel's result into the buffer of its `outs` operand unless that buffer is read-only or still read later; then it copies first.
Output	`6-bufferized.mlir`: three copies, one workspace, an `info` result.
Theory	FG §10.3 *Bufferization and destination-passing style*; FG §2.7 *Value semantics vs reference/memory semantics in IRs.*

The core of the bufferized function (the workspace protocol at its start and the `scf.if` nesting of the `info` checks are left out; see Section 6.4 for both):

```
%view = memref.view %25[%c0][%arg0, %arg0] : memref<?xi8> to memref<?x?xf64>
memref.copy %arg3, %view : memref<?x?xf64> to memref<?x?xf64>          // copy of M
%info = ulm_lamk.potrf uplo(lower) outs(%view : memref<?x?xf64>)        // L1 in the copy
...                                                                     // if info == 0:
%view_1 = memref.view %25[%4][%arg1, %arg1] : memref<?xi8> to memref<?x?xf64>
%view_2 = memref.view %25[%9][%arg0, %arg1] : memref<?xi8> to memref<?x?xf64>
ulm_lamk.gecopy trans(trans) ins(%arg2 : memref<?x?xf64>) outs(%view_2 : memref<?x?xf64>)
ulm_lamk.trsm side(left) uplo(lower) trans(none) diag(non_unit) alpha(%cst)
    ins(%view : memref<?x?xf64>) outs(%view_2 : memref<?x?xf64>)       // W
ulm_lamk.syrk uplo(lower) trans(trans) alpha(%cst) beta(%cst_0)
    ins(%view_2 : memref<?x?xf64>) outs(%view_1 : memref<?x?xf64>)     // S
%info_3 = ulm_lamk.potrf uplo(lower) outs(%view_1 : memref<?x?xf64>)   // L2
...                                                                     // if info_3 == 0:
%view_4 = memref.view %25[%13][%arg0] : memref<?xi8> to memref<?x1xf64>
memref.copy %arg4, %view_4 : memref<?x1xf64> to memref<?x1xf64>        // copy of y
ulm_lamk.trsv ... ins(%view : memref<?x?xf64>) outs(%view_4 : memref<?x1xf64>)        // z
```

```
ulm_lamk.gemv trans(trans) ... ins(%view_2, %view_4 : ...) outs(%arg5 : memref<?x1xf64>)
ulm_lamk.trsv uplo(lower) trans(none) ... ins(%view_1 : ...) outs(%arg5 : memref<?x1xf64>)
ulm_lamk.trsv uplo(lower) trans(trans) ... ins(%view_1 : ...) outs(%arg5 : memref<?x1xf64>)
```

Why three copies. `potrf` overwrites its matrix, `trsm` and `trsv` their right-hand sides. M, X and y are `ulm_lam.in`, so `ulm-lam-results-to-out-params` gave them `bufferization.writable = false` (`lib/UlmLamResultsToOutParams.cpp:344`), and One-Shot Bufferize must not write into them: it copies M and y. For X the copy was already in the IR (`gecopy`, a transposing copy that `trsm` then overwrites), so no further copy is needed. With `InOut(...)` parameters the caller allows overwriting; `examples/gls/check.sh` verifies that `potrf` then works on the caller's M and only X is copied.

One value, two uses, one buffer. W lives in `%view_2`, written once by `trsm` and read by `syrk` and `gemv`. SSA use counts are not copies: a value with two uses needs a second buffer only if one use would overwrite it while the other still reads it.

The result. β is computed directly in the caller's output buffer `%arg5`: `gemv` writes it, the two `trsv` update it in place.

The workspace. All temporaries are views of one byte buffer `%25` at offsets computed from n and m, each rounded up to 64 bytes (Figure 6.1). The caller can pass that buffer, query its size first (LAPACK's `lwork` convention), or let the function allocate it.

`info`. Each factorization is a numbered check. If `potrf` reports failure, the rest of the block does not run and the function returns the check's number. In C++ this is the return value of `gls`; in Julia, an `InfoError`.

> **Theory (FG §10.3, FG §2.7, FG §1.4).** Tensors have value semantics, memrefs reference semantics, FG §2.7 *Value semantics vs reference/memory semantics in IRs.* Bufferization chooses, for every DPS result, between reusing the destination's buffer and copying, FG §10.3 *Bufferization and destination-passing style.* The in-place condition (no other live reader of the old value, and the buffer may be written) is a uniqueness condition, as FG §1.4 *Uniqueness types vs linear types* explains; here it is decided by an analysis, not by a type system. Chapter 6 goes into detail.

1.9 The kernel sequence in C++

`convert-ulm-lamk-to-ulmblas` lowers the memref program to the EmitC dialect, and `mlir-translate --mlir-to-cpp` prints C++. With the Julia backend the calls go to CBLAS (the C interface to BLAS) and LAPACK through Julia's libblastrampoline, which forwards them to OpenBLAS here. The calls of `7-gls.cpp`, in order (`v1` $= n$, `v2` $= m$, `v3` $= X^{\mathsf{T}}$ row-major, `v4` $= M$, `v5` $= y$, `v6` $= \beta$):

```
ulm_lam_copy(v31, v4, v32);                                          // M -> workspace
int64_t v33 = ulm_lam_cblas::potrf<std::ptrdiff_t>(true, v1, v31, v1, 1);
ulm_lam_gecopy(v1, v2, v3, 1, v1, v41, v2, 1);                       // X (transposing copy)
(void)ulm_lam_cblas::trtrs<std::ptrdiff_t>(
    true, false, false, false, v1, v2, v31, v1, 1, v41, v2, 1);      // W = L1^-1 X
ulm_lam_cblas::sylrk<std::ptrdiff_t>(v2, v1, v9, v41, 1, v2,
                                     v10, v39, v2, 1);               // S = W^T W (lower)
int64_t v42 = ulm_lam_cblas::potrf<std::ptrdiff_t>(true, v2, v39, v2, 1);
ulm_lam_copy(v45, v5, v46);                                          // y -> workspace
ulm_lam_cblas::trlsv<std::ptrdiff_t>(v1, false, v31, v1, 1, v45, 1);  // z = L1^-1 y
ulm_lam_cblas::gemv<std::ptrdiff_t>(v2, v1, v9, v41, 1, v2,
                                    v45, 1, v10, v6, 1);             // beta = W^T z
ulm_lam_cblas::trlsv<std::ptrdiff_t>(v2, false, v39, v2, 1, v6, 1);   // beta = L2^-1 beta
ulm_lam_cblas::trusv<std::ptrdiff_t>(v2, false, v39, 1, v2, v6, 1);   // beta = L2^-T beta
```

Two details show what lowering does with the remaining structure. The transposition flags are gone: a

transposed operand is the same pointer with swapped strides (`v39, 1, v2` versus `v39, v2, 1`), so L_2^{T} is passed as an *upper* triangular matrix to `trusv`. And the matrix properties are gone entirely; what is left of them is the choice of routines and their `uplo` arguments.

Kernel	Library	Flops	Computes
`potrf`	LAPACK	$n^3/3$	$M = L_1 L_1^{\mathsf{T}}$, in a copy of M
`trsm` (`trtrs`)	BLAS 3	$n^2 m$	$W = L_1^{-1} X$, in a copy of X
`syrk`	BLAS 3	nm^2	$S = W^{\mathsf{T}} W$, lower triangle
`potrf`	LAPACK	$m^3/3$	$S = L_2 L_2^{\mathsf{T}}$, in place
`trsv`	BLAS 2	n^2	$z = L_1^{-1} y$, in a copy of y
`gemv`	BLAS 2	$2nm$	$\beta \leftarrow W^{\mathsf{T}} z$
`trsv`, `trsv`	BLAS 2	$2m^2$	$\beta \leftarrow L_2^{-\mathsf{T}} L_2^{-1} \beta$

1.10 A numerical example

Small inputs make every step checkable by hand. Take $n = 4$, $m = 2$,

$$X = \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{pmatrix}, \quad M = \begin{pmatrix} 4 & 2 & 0 & 0 \\ 2 & 5 & 2 & 0 \\ 0 & 2 & 5 & 2 \\ 0 & 0 & 2 & 5 \end{pmatrix}, \quad y = \begin{pmatrix} 1 \\ 2 \\ 4 \\ 4 \end{pmatrix}.$$

M is SPD (eigenvalues between 1.58 and 8.13) with Cholesky factor $L_1 = 2$ on the diagonal and 1 below it, so every step up to S is exact in binary floating point:

$$W = L_1^{-1} X = \begin{pmatrix} 1/2 & 0 \\ 1/4 & 1/2 \\ 3/8 & 3/4 \\ 5/16 & 9/8 \end{pmatrix}, \quad S = W^{\mathsf{T}} W = \begin{pmatrix} 141/256 & 97/128 \\ 97/128 & 133/64 \end{pmatrix}, \quad X^{\mathsf{T}} M^{-1} y = \begin{pmatrix} 363/256 \\ 375/128 \end{pmatrix}.$$

Expected. In exact rational arithmetic,

$$\beta = \left(\frac{93}{73}, \frac{69}{73}\right) = (1.273972602739726\ldots,\ 0.945205479452054\ldots).$$

Obtained. `examples/gls.jl` prints:

```
==== exact β (rational): Rational{BigInt}[93//73, 69//73]
==== exact β (Float64):  [1.273972602739726, 0.9452054794520548]
==== ulm.lam β: [1.2739726027397262, 0.9452054794520546]
==== Julia expert  (X'*(M\X)) \ (X'*(M\y)): [1.2739726027397262, 0.9452054794520547]
==== Julia formula inv(X'*inv(M)*X)*X'*inv(M)*y: [1.2739726027397262, 0.9452054794520547]
==== max abs error vs exact: 2.220446049250313e-16
```

ulm.lam's result differs from the correctly rounded β by one unit in the last place in each component, and from Julia's two evaluations in the second component by one unit. All three are correct to working accuracy; they differ because they round differently, not because one is wrong.

Failing and rejected inputs. The same program with inputs that violate its assumptions (all from `outputs/gls.out`):

Input	Result	Origin of the check
$M_{33} = -1$: symmetric, indefinite	`InfoError: gls: info 1: Cholesky factorisation of v4: not positive definite`	run time: `potrf` of M
X with two equal columns, declared `full_rank`	`InfoError: gls: info 2: Cholesky factorisation of a computed matrix: not positive definite`	run time: `potrf` of S
y of length 3	`LamError: gls: y: size :n = 3, but 4 from an earlier argument`	run time: the Julia runtime compares sizes before the call
$M_{12} = 100$ (Julia's upper triangle), declared `spd`	`InfoError: ... info 1 ...`	run time, by luck: `potrf` reads the triangle that was changed
$M_{21} = 100$ (Julia's lower triangle), declared `spd`	β of `Symmetric(M, :U)`; no error	not established: symmetry is never checked, one triangle is ignored
`Mat(:n,:m) * Mat(:n,:m)` in a `@lam` body	`LamError: product of nxm and nxm: m columns but n rows`	at trace time, by the frontend (Chapter 4)

Two remarks. The messages name `v4`, the fourth C parameter, where `@code_lam` says `M`: the description comes from the C++ interface comment. And the rank-deficient X is caught only because S became exactly singular here. A nearly rank-deficient X can pass the factorization and give an inaccurate β, or fail although X has full rank (Section 7.4).

1.11 The chapter in one table

Step	MLIR mechanism	Theory	FG
tracing	text emitted by overloaded Julia operators	tracing embedded DSL	FG §3.7
facts on matrices	builtin tensor type with an attribute in its encoding	refinement types	FG §1.7
normal form	`OpRewritePatterns`, greedy driver	rewriting, termination	FG §3.4, FG §3.5
$X^{\mathsf{T}}M^{-1}X$ is SPD	`InferTypeOpInterface` per op	type inference; abstract interpretation	FG §1.10, FG §5.2
kernel selection	custom pass, dynamic program over leaves	instruction selection	FG §4.5, FG §3.3
copies, in place	DPS ops, One-Shot Bufferize	value vs reference semantics; uniqueness	FG §10.3, FG §2.7, FG §1.4
failed factorization	`info` results, `scf.if`	run-time check	—
C++	dialect conversion to EmitC	lowering, legalization	FG §3.3, FG §4.6

CHAPTER 2

MLIR Provides the Infrastructure, ulm.lam Provides the Meaning

Answer. MLIR [1] has an open, extensible type system. A dialect defines its own types, attributes and operations, and states the rules that they must satisfy; MLIR runs those rules at defined points and rejects IR that breaks them. MLIR does not come with any particular type *discipline*: not linear types, not dependent types, not refinement types, not an effect system. Each of those needs a semantics and an enforcement mechanism that someone has to design and implement. In short: *MLIR provides the infrastructure for representing domain-specific types and enforcing rules about operations. The meaning of those types, and the strength of the guarantees they provide, comes from the dialect and its implementation.*

The Field Guide describes the infrastructure side in FG §3.1 *Dialects and extensible operation sets* and FG §3.2 *Traits and interfaces*, and the type disciplines in FG Chapter 1 *Types as Static Resource Reasoning*. This chapter shows, with ulm.lam, which of the two you are looking at in each case.

2.1 Who provides what

Concern	MLIR provides	ulm.lam implements
types	builtin `tensor` and `memref` with an *encoding* slot; ODS (Operation Definition Specification, in TableGen) for new types and attributes	no new type; one attribute, `#ulm_lam.props<...>`, used as tensor encoding
operations	ODS: operands, results, type constraints, assembly format	`ulm_lam`: 7 ops (`mul`, `add`, `scale`, `transpose`, `inverse`, `identity`, `assume`); `ulm_lamk`: 17 kernel ops and `call`
verification	hooks (`verify()`, `verifyEncoding`), generated operand checks, `InferTypeOpInterface`'s result check; run after parsing and (by default) after each pass	the rules: inner dimensions, squareness, contradictory properties, DPS result types
inference	`InferTypeOpInterface` and its compatibility hook `isCompatibleReturnTypes`	the property rules (`mulProperties`, ...) and what "compatible" means
effects	`MemoryEffectOpInterface`, the `Pure` trait; generic passes (CSE, DCE (dead code elimination), LICM (loop-invariant code motion)) that consult them	which ops are pure; which operands a kernel reads and writes
rewriting	pattern classes, the greedy driver, the dialect conversion framework	the normal form, kernel selection, the lowering to EmitC
memory	One-Shot Bufferize; `DstBufferizableOpInterfaceExternalModel` for DPS ops	DPS kernel ops and their bufferization models; the workspace and `info` passes

Table 2.1: The division of labor between MLIR and ulm.lam.

Read the table column by column. Everything on the left is generic and knows nothing about matrices. Everything on the right is a decision of ulm.lam. When this text says that "the compiler knows that S is

SPD", the knowledge is in the right column: MLIR stores the attribute, calls the inference hook and runs the verifier, but the meaning of `spd` and the rule that derives it are ulm.lam's C++.

2.2 A property set as a tensor encoding

Input	a type written as `tensor<?x?xf64, #ulm_lam.props<spd>>`.
Representation	a builtin `RankedTensorType` whose encoding is a `PropertiesAttr`, a 32-bit set of named facts.
Rule	`verifyEncoding`, called by MLIR whenever such a type is built or parsed.
Output	the type, or a diagnostic.
Theory	a refinement of the base type by a predicate, FG §1.7 *Refinement types*.

The declaration (`include/ulm_lam/UlmLamAttrs.td:37`, cases shortened):

```
def UlmLam_PropSquare     : I32BitEnumAttrCaseBit<"square",       0>;
def UlmLam_PropSymmetric  : I32BitEnumAttrCaseBit<"symmetric",    1>;
def UlmLam_PropSPSD       : I32BitEnumAttrCaseBit<"spsd",         2>;
def UlmLam_PropSPD        : I32BitEnumAttrCaseBit<"spd",          3>;
...
def UlmLam_PropFullRank   : I32BitEnumAttrCaseBit<"full_rank",   12>;
def UlmLam_PropLowerStored: I32BitEnumAttrCaseBit<"lower_stored",13>;
def UlmLam_PropUpperStored: I32BitEnumAttrCaseBit<"upper_stored",14>;
def UlmLam_PropDiagonal   : I32BitEnumAttrCaseGroup<"diagonal",
                              [UlmLam_PropLowerTri, UlmLam_PropUpperTri]>;
...
// Used as a builtin tensor encoding:  tensor<?x?xf64, #ulm_lam.props<spd>>
def UlmLam_PropsAttr : EnumAttr<UlmLam_Dialect, UlmLam_Properties, "props",
    [DeclareAttrInterfaceMethods<VerifiableTensorEncoding>]> {
  let assemblyFormat = "< $value >";
  ...
}
```

Two MLIR mechanisms carry it: `EnumAttr` over an `I32BitEnumAttr` generates the attribute class, its parser and printer; the interface `VerifiableTensorEncoding` makes MLIR call a verifier when a tensor type with this encoding is created. The verifier is ulm.lam's (`lib/UlmLamDialect.cpp:145`):

```
LogicalResult PropertiesAttr::verifyEncoding(ArrayRef<int64_t> shape, Type,
    function_ref<InFlightDiagnostic()> emitError) const
{
    P p = closure(getValue(), shape);
    if (shape.size() != 2)
        return emitError() << "#ulm_lam.props needs a rank-2 tensor";
    if (has(p, P::zero) && has(p, P::full_rank))
        return emitError() << "#ulm_lam.props: zero contradicts full rank";
    ...
    if (has(p, P::square) && !ShapedType::isDynamic(shape[0]) &&
        !ShapedType::isDynamic(shape[1]) && shape[0] != shape[1])
        return emitError() << "#ulm_lam.props: square property on a "
                           << shape[0] << "x" << shape[1] << " tensor";
    return success();
}
```

It first closes the set under implication (so `spd` brings `square`), then checks the closure against itself and against the static shape. Two rejected types (`outputs/verify.out`):

```
### v3-square-props.mlir
v3-square-props.mlir:1:53: error: #ulm_lam.props: square property on a 3x4 tensor
func.func @f(%a: tensor<3x4xf64, #ulm_lam.props<spd>>) {
### v4-zero-fullrank.mlir
v4-zero-fullrank.mlir:1:68: error: #ulm_lam.props: zero contradicts full rank
func.func @f(%a: tensor<?x?xf64, #ulm_lam.props<zero, non_singular>>) {
```

`spd` on a 3×4 tensor is rejected because SPD implies square; `zero` with `non_singular` because non-singular implies full rank. Both are contradictions *between the facts and the shape*, decided without looking at any data. `tensor<4x4xf64, #ulm_lam.props<spd>>` is accepted for every 4×4 matrix, SPD or not: a type cannot be wrong about entries it never sees.

2.3 Operation verifiers: structural rules

Each op class may define `verify()`. MLIR calls it after parsing and, with the default pass-manager settings, after every pass. For `ulm_lam.mul` (`lib/UlmLamDialect.cpp:514`):

```
LogicalResult MulOp::verify()
{
    if (getFactors().size() < 2)
        return emitOpError("needs at least two factors");
    for (unsigned i = 0; i + 1 < getFactors().size(); ++i) {
        Type a = getFactors()[i].getType(), b = getFactors()[i + 1].getType();
        if (!dimsCompatible(cols(a), rows(b)))
            return emitOpError("factor ") << i << " has " << cols(a)
                   << " columns but factor " << i + 1 << " has " << rows(b) << " rows";
    }
    return success();
}
```

`dimsCompatible(a, b)` is true if either is dynamic or both are equal. Accepted and rejected (`outputs/verify.out`):

```
### v1-mul-static.mlir
v1-mul-static.mlir:2:8: error: 'ulm_lam.mul' op factor 0 has 4 columns but factor 1 has 5 rows
### v2-mul-dynamic.mlir
accepted
### v5-inverse-nonsquare.mlir
v5-inverse-nonsquare.mlir:2:8: error: 'ulm_lam.inverse' op operand is not square
### v9-gemm.mlir
v9-gemm.mlir:3:8: error: 'ulm_lamk.gemm' op shapes of op(A), op(B) and C do not match
### v10-gemm-dps.mlir
v10-gemm-dps.mlir:3:8: error: 'ulm_lamk.gemm' op result 0 must have the type of its init
```

`v2` multiplies two `tensor<?x?xf64>`: the verifier cannot know whether the inner sizes agree, so it accepts. Chapter 4 shows where dynamic sizes *are* checked. `v10` violates a DPS rule: a kernel's tensor result must have the type of the operand it overwrites (`verifyDps`, `lib/UlmLamDialect.cpp:826`).

> **Not the same thing.** An operation verifier checks the IR, not the data. It sees types, attributes and the structure of the program, and it runs at compile time. It cannot tell whether the 4×4 array that a caller passes is SPD. In ulm.lam that question is answered only by `potrf` at run time, and only for the matrices that get factorized.

2.4 Result types must be justified

Input	an op whose result type is written out, possibly with properties.
Representation	`ulm_lam` ops implement `InferTypeOpInterface`.
Rule	MLIR's generated verifier infers the result type from the operands and calls `isCompatibleReturnTypes(inferred, declared)`; ulm.lam defines compatible as "same shape up to `?`, same element type, and every declared property is implied by the inferred ones".
Output	a declared type may be weaker than the inferred one, never stronger.
Theory	subtyping by implication, FG §1.7 *Refinement types*; checking against an annotation, FG §1.10 *Type inference beyond Hindley–Milner: bidirectional and constraint-based.*

The compatibility relation (`lib/UlmLamDialect.cpp:359`):

```
/// Shapes compatible, element types equal, properties of actual implied
/// by inferred.
bool ulmLamCompatible(TypeRange inferred, TypeRange actual)
{
    ...
        P have = closure(propertiesOf(it), it.getShape());
        if (!has(have, propertiesOf(at)))
            return false;
    ...
}
```

The test is the pair `v6` and `v7`: the same Gram product $A^{\mathsf{T}}A$ of a 4×3 matrix, with the result declared `spd`. Without `full_rank` on A, only SPSD can be inferred, and the verifier rejects the claim:

```
### v6-overclaim.mlir
v6-overclaim.mlir:3:8: error: 'ulm_lam.mul' op inferred type(s) 'tensor<3x3xf64, #ulm_lam.props<spsd>>'
are incompatible with return type(s) of operation 'tensor<3x3xf64, #ulm_lam.props<spd>>'
v6-overclaim.mlir:3:8: error: 'ulm_lam.mul' op failed to infer returned types
### v7-overclaim-ok.mlir
accepted
```

With `full_rank` on A (`v7`), SPD is inferable and the same declaration is accepted. So inside a function, a property on a result type is not an assumption: it is a claim that the verifier checks against the inference rules. The only way to state more than inference derives is the dedicated op `ulm_lam.assume`, which has no inference hook and is accepted with any properties (`v8` in `outputs/verify.out`). Section 3.2 discusses it.

> **Theory (FG §1.7, FG §1.10).** In a refinement type system, a value of type $\{v : B \mid p\}$ may be used where $\{v : B \mid q\}$ is expected if $p \Rightarrow q$. Here B is the tensor type, p the inferred property set and q the declared one, and $p \Rightarrow q$ is decided by closing p under the implication rules and testing bit-set containment, not by an SMT (satisfiability modulo theories) solver: with fifteen atoms and a fixed set of Horn-like implications the check is a few bit operations. Comparing a synthesized type against a written one is the *checking* direction of bidirectional typing.

2.5 Operation traits are not matrix properties

Both are called "properties" in casual speech, and both are written in ODS. They live in different worlds.

Discipline (FG)	In ulm.lam	Status
refinement types (FG §1.7)	property sets on tensor types; results checked by implication; parameters and `assume` trusted	**present**, with a finite predicate language
type inference (FG §1.10)	synthesis of result types per op, one pass in program order	**present**
indexed / dependent types (FG §1.8)	static shapes checked; run-time sizes named in the Julia frontend and in `ulm_lam.dims`; no type-level terms	**analogy** (Chapter 4)
effect systems (FG §1.6)	per-op memory effects, used by generic passes; no effects in function types, no effect polymorphism	**partly**: effect annotations, not an effect type system
linear / affine types (FG §1.2, FG §1.3)	none: SSA values may be used any number of times	**absent** (Chapter 5)
uniqueness (FG §1.4)	the in-place condition of bufferization, decided by an analysis	**analogy** (Chapter 6)
region types (FG §1.5)	none; MLIR regions are nested code, the workspace is one arena per call	**absent**; analogy only
typestate (FG §1.9)	"factorized" is tracked by a memo table in kernel selection, not in types	**absent**

Table 2.2: Type disciplines of FG Chapter 1 and their status in ulm.lam.

	Op traits and interfaces	Matrix properties
example	`Pure`, `Commutative`, `DestinationStyleOpInterface`	`spd`, `lower_tri`, `full_rank`
describes	an *operation class* (all `ulm_lam.mul` ops)	a *value* (this matrix)
where	the op's C++ class, via ODS	the value's type (encoding)
read by	generic MLIR passes: CSE, DCE, LICM, canonicalizer, bufferization	only ulm.lam's own rules and passes
MLIR understands it	yes: that is its purpose	no: an opaque attribute

`ulm_lam.add` has the trait `Commutative` (`include/ulm_lam/UlmLamOps.td:63`); `ulm_lam.mul` does not, because matrix products do not commute. The trait states a fact about the *operation* that a generic canonicalizer may use (to sort operands). A matrix being SPD states a fact about the *operand*, and no upstream pass will ever read it.

> **Not the same thing.** MLIR operation traits are not mathematical matrix properties. The trait `Pure` on `ulm_lam.inverse` says that the operation has no memory effects and may be executed speculatively; it says nothing about whether the operand is invertible. Section 5.4 shows what follows from that difference.

2.6 What is present, and what is related by analogy

Table 2.2 is the short answer to "which theoretical type systems does ulm.lam have?". Three entries need a remark.

Declared but not implemented. The ODS of `ulm_lam.assume` has a unit attribute `checked`, documented as "verified at run time (e.g. symmetry test, or success of the Cholesky factorisation for spd)" (`include/ulm_lam/UlmLamOps.td:135`). At commit `4185542` neither frontend sets it and no pass generates a check from it; the only code that reads it is a normalization pattern that leaves checked

assumptions alone (`lib/UlmLamPasses.cpp:79`). Likewise, the pass description in `include/ulm_lam/UlmLamPasses.td` lists kernel-selection strategies `search` and `egraph`, a `cost-model` option and a pass `convert-ulm-lamk-to-linalg`; that file is documentation (the passes are registered by hand in C++), and the implemented pass rejects every strategy but `gmc` (`lib/UlmLamPasses.cpp:2295`).

Interfaces that nothing calls. `ulm_lam` and `ulm_lamk` ops implement two interfaces of ulm.lam's own, `PropertyInferenceOpInterface` and `KernelCostOpInterface` (`include/ulm_lam/UlmLamInterfaces.td:11`). No pass at this commit calls them: inference goes through MLIR's `InferTypeOpInterface`, and kernel selection computes flop counts inline (Section 8.1). They are the extension points of a design, not mechanisms of the current compiler.

Effects are partly present. MLIR's effect interface is a real effect annotation per op, and generic passes use it soundly (Chapter 5). What the Field Guide calls an effect *system*, effects in function types that compose by union and that a type checker verifies, does not exist in MLIR or in ulm.lam.

CHAPTER 3

Assumed, Verified, Inferred, Checked: Where Facts Come From

Answer. ulm.lam proves nothing about the entries of a matrix. Facts enter in two places only, the parameter types and `assume`, and are trusted there. Inference rules derive new facts from them; these are theorems about real matrices, valid if the declarations are. Verifiers check that the IR is consistent with the rules, never with the data. At run time the Julia (or Python) runtime checks the array sizes, and each factorization checks the one fact it depends on: `potrf` detects a matrix that is not positive definite in floating point, `getrf` an exactly singular one. Symmetry of an input, full column rank and conditioning are not checked.

3.1 The facts and how they imply each other

A property set is a conjunction of atoms. Its meaning is fixed by one C++ function, `closure` (`lib/UlmLamDialect.cpp:97`), which adds everything that the given atoms imply (Figure 3.1).

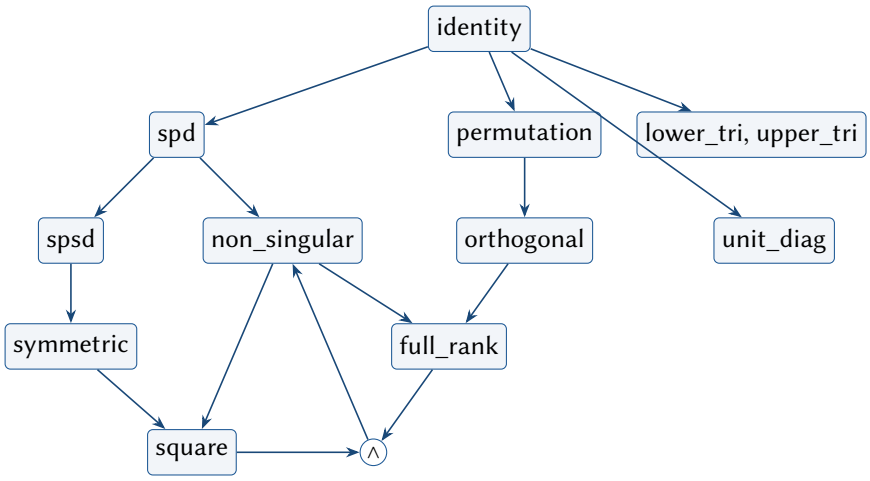


Figure 3.1: The implications applied by `closure`. The circle is a conjunction: square and full rank imply non-singular. A static square shape adds `square`. `lower_tri` together with `upper_tri` is printed as `diagonal`.

Three conventions make this a sound basis for reasoning:

- **Absence means unknown.** A set without `spd` says nothing about definiteness; it does not say "indefinite". The empty set (printed as no encoding) means "some matrix".
- **Dropping a fact is always sound.** A value with `{spd}` may be used where `{symmetric}` or `{}` is expected. Rewrites use this to keep result types unchanged when they find a better-typed replacement (they insert an `assume` that only drops facts, `replaceKeepingType`, `lib/UlmLamDialect.cpp:567`).
- **Two bits are not facts about the value.** `lower_stored` and `upper_stored` say that only one triangle of the *memory* is meaningful. They are never inferred and never propagated to a result,

and dropping them would be wrong; the comment in `include/ulm_lam/UlmLamAttrs.td:10` spells this out.

> **Theory (FG §5.1, FG §5.2).** The property sets, ordered by implication, form a finite lattice; "absence means unknown" makes each set an over-approximation of the matrices it describes, which is the abstraction of [9] and FG §5.2 *Abstract interpretation: Galois connections, widening, narrowing.* `closure` plays the role of a reduction (it adds what the other atoms imply, without changing the set of matrices described), and each op's rule is a sound abstract transfer function. Because a function without loops is a DAG (directed acyclic graph) of SSA values, one pass in program order computes the result; no fixpoint iteration (FG §5.1 *Lattices, monotone frameworks and fixpoints (a sharper recap)*) is needed. Loops are the exception, discussed next.

Loops: facts are re-declared, not inferred. A loop-carried matrix enters the loop body as a block argument. The Julia frontend gives every loop-carried value a type without properties (`plainvalue` in `loop`, `julia/UlmLam/src/frontend.jl:408`), and `ulm-lam-infer-properties` refines op results only, not block arguments. A fact that holds in every iteration therefore has to be re-declared, as the Kalman filter in the README does:

```
loop(steps, x0, P0) do i, x, P
    P = assume(P, :symmetric)
    ...
```

Proving that P stays symmetric would need a fixpoint over the loop: start from the facts of P_0, apply the body's transfer functions, and intersect with the facts at the loop head until nothing changes. That is exactly the iterative dataflow of FG §5.1; ulm.lam does not implement it and asks the programmer instead.

3.2 Declared facts: parameters and `assume`

Input	`assume(x, :spd)` in Julia (`lam.assume(x, "spd")` in Python).
Representation	`%y = ulm_lam.assume %x : tensor<...> → tensor<..., #ulm_lam.props<spd>>`.
Rule	none: the result has the properties of its type. The verifier only checks that shape and element type are unchanged.
Output	a value that later rules treat as SPD.
Theory	an unchecked cast into a refinement type; the trusted boundary of FG §1.7 *Refinement types.*

The ODS description (`include/ulm_lam/UlmLamOps.td:135`) says: "Properties the operand does not have are *assertions*: with `checked` they are verified at run time [...], otherwise trusted. This is the only op that may add facts that inference cannot derive." At this commit the `checked` variant is not implemented (Section 2.6); the Julia docstring of `assume` (`julia/UlmLam/src/frontend.jl:351`) states the actual behavior: "Trusted; a factorisation that finds it wrong returns info > 0 (`InfoError`)."

What a declaration changes. The same GLS formula with different declarations (`outputs/variants.out`, shortened):

```
gls_nofr(X::n×m, M::n×n spd, y::n)                # X without :full_rank
  potrf    L1 ← chol(M)
  copy     T1 ← X
  trsm     T1 ← L1⁻¹ T1
  syrk     T2 ← T1ᵀ T1                        one triangle only
```

```
  symm.    T2 ← sym(T2)                    copy one triangle to the other
  getrf    LU1 ← lu(T2)                    P T2 = L U, LU
  ...
gls_assume(X::n×m, M::n×n spd, y::n)              # S = assume(X' * inv(M) * X, :spd)
  potrf    L1 ← chol(M)
  copy     T1 ← X
  trsm     T1 ← L1⁻¹ T1
  syrk     T2 ← T1ᵀ T1                     one triangle only
  potrf    L2 ← chol(T2)                   T2 = L2 L2ᵀ, Cholesky
  ...
```

Without `full_rank`, $S = X^{\mathsf{T}}M^{-1}X$ can only be inferred SPSD, so S^{-1} is not mapped to Cholesky: kernel selection completes the symmetric matrix (`sycomplete`) and factorizes it by LU. With the assumption S is SPD again and the Cholesky path returns. All variants compute the same β for the example data; they differ in cost and in what they check.

A false declaration. M declared SPD but not symmetric ($M_{21} = 100$ in Julia's lower triangle, `outputs/variants.out`):

```
ulm.lam: [1.2739726027397262, 0.9452054794520546]
Julia with Symmetric(Mns, :U): [1.2739726027397262, 0.9452054794520547]
Julia with Mns as given:       [1.2726368159203976, 0.9552238805970151]
```

`potrf` reads one triangle (in Julia's terms the upper one: the IR holds the transpose) and never looks at the other, so ulm.lam computes the GLS estimate for the symmetric matrix that the upper triangle defines. No error is reported; the result is the answer to a different problem. The origin of symmetry is declared and its check is not established.

> **Not the same thing.** An assumption is not a run-time check. `assume(A, :spd)` and the parameter type `Mat(:n, :n, :spd)` make the compiler *act* on SPD; they do not test it. A Cholesky factorization of A does test positive definiteness of the stored triangle, as a side effect of computing the factor. Whether such a factorization happens depends on the formula: a matrix that is declared SPD but only multiplied (Ax) is never tested.

3.3 Inferred facts: the rules

Table 3.1 lists every inference rule, read from `lib/UlmLamDialect.cpp:192` and `lib/UlmLamDialect.cpp:900`. All are facts of linear algebra over $\mathbb{R}$; each is applied to the *closures* of the operand sets.

Two of the rules show how much depends on details:

- `scale` keeps SPD only for a positive *constant*. For a scalar known only at run time (`Float` parameter), the sign is unknown, so $\sigma^2 K$ loses SPD. This is why the README writes `assume(K + s2 * I, :spd)` for a Gaussian process.
- The palindrome rule compares operands by *identity*, not by value: $X^{\mathsf{T}}M^{-1}X$ is recognized because both outer factors are the same SSA value, once transposed. Two different values that happen to hold equal matrices are not recognized. The interface description says so ("Operand *identity* matters, not only operand properties", `include/ulm_lam/UlmLamInterfaces.td:11`).

Why inferred facts matter. They choose the kernels. Table 3.2 summarizes `outputs/variants.out`.

> **Not the same thing.** Property inference is not a general theorem prover. It applies a fixed set of rules, each local to one op, to a fixed vocabulary of fifteen atoms. It cannot use facts about sizes ($n \geq m$), cannot relate two different values, and cannot prove anything that needs algebra across

Op	Result facts
`mul` $F_1 \cdots F_k$	the facts shared by all factors among `lower_tri`, `upper_tri`, `unit_diag`, `orthogonal`, `permutation`, `non_singular`, `identity`; `zero` if any factor is zero; `full_rank` for NA or AN with N non-singular and A full rank (two factors); and the palindrome rule: $Z^\mathsf{T}Z$ SPSD (SPD if Z has full column rank), $Z^\mathsf{T}MZ$ symmetric / SPSD / SPD if M is
`add` $A_1 + \cdots + A_k$	the facts shared by all terms among `symmetric`, `spsd`, `lower_tri`, `upper_tri`; SPD if all are SPSD and one is SPD
`scale` αA	triangularity, symmetry, zero; SPD/SPSD if α is a *constant* > 0; rank facts if $\alpha \neq 0$; zero if $\alpha = 0$
`transpose`	everything, with `lower_tri` and `upper_tri` exchanged; storage bits dropped
`inverse`	triangularity, unit diagonal, symmetric, SPD, orthogonal, permutation, identity, non-singular, full rank, square
`identity`	`identity`
`assume`	the properties of its result type (declared)
`ulm_lamk.trsm`	`full_rank` if the right-hand side has full rank
`ulm_lamk.syrk`	`spsd`; `ulm_lamk.gemmtr`: `symmetric`
other kernels	nothing (`potrf`'s result holds a factor, not the matrix)

Table 3.1: The inference rules of ulm.lam at commit `4185542`.

Declarations	Inferred for S	Kernels for M^{-1} and S^{-1}
X full rank, M SPD	SPD	`potrf`, `potrf`; `syrk` for S
X general, M SPD	SPSD	`potrf`; `syrk`, `sycomplete`, `getrf`
X full rank, M symmetric	symmetric	`getrf`, `getrf`; `gemmtr` for S
X full rank, M general	none	`getrf`, `getrf`; `gemm` for S
X general, M SPD, `assume(S, :spd)`	SPD (declared)	`potrf`, `potrf`; `syrk`

Table 3.2: The same formula under different declarations.

ops, for example that $A + A^\mathsf{T}$ is symmetric (`add` only keeps facts that all terms share). Where a rule does not apply, the fact is unknown, and the program is still compiled, just with more general kernels.

3.4 Facts established by analyses

Some facts are not about matrices but about the program. They are found by analyses that inspect the IR:

- **Which size a dynamic dimension has.** `dimOf` (`lib/UlmLamDialect.cpp:1269`) follows a value back through ulm.lam ops, DPS destinations, loops, branches and calls to the function parameter that `ulm_lam.dims` names. Section 4.3.
- **Whether an earlier product may be reused.** Kernel selection reuses W only if its definition properly dominates the use (`dom.properlyDominates`, `lib/UlmLamPasses.cpp:1080`).
- **Whether an op is loop invariant.** LICM moves an op out of a loop if its operands are defined outside and it is pure (Section 5.3).
- **Whether a write may be in place.** One-Shot Bufferize's read-after-write conflict analysis (Chapter 6).

These are analysis facts: derived from the program text, sound for all inputs, and independent of the matrix entries.

3.5 Facts checked at run time

- **Array sizes**, in the Julia runtime before the C call (`bind_sizes!`, `julia/UlmLam/src/runtime.jl:326`): every run-time size symbol must have one value across all arguments, and every fixed size must match. The generated C function takes the sizes as arguments and does not check them.
- **Positive definiteness**, by `potrf`: LAPACK reports info $= i > 0$ if the leading minor of order i is not positive definite *in floating point*. The compiled function returns the number of the check (Section 5.5).
- **Exact singularity**, by `getrf`: info $= i > 0$ if U_{ii} is exactly zero. A nearly singular matrix passes.
- **Workspace size**, if the caller provides the workspace: info $= -i$ when it is too small.

What is not checked anywhere: that a matrix declared symmetric (or SPD) is symmetric; that X has full column rank or that $n \geq m$; that a declared triangular matrix has zeros in its other triangle (the kernels never read it); the conditioning of anything.

3.6 Three ways to know that a matrix is SPD

The task statement of this companion contrasts three situations. With the GLS example they are:

Situation	Origin	Consequence if the fact is false
`M::Mat(:n, :n, :spd)`	declared, trusted	`potrf` of M fails (`info 1`) if M is not positive definite; wrong β, no error, if M is not symmetric
$S = X^{\mathsf{T}}M^{-1}X$ is SPD	inferred from M SPD and X full column rank	cannot be false over $\mathbb{R}$ if the premises hold; in floating point `potrf` of S may still fail (Section 7.4)
`potrf` returns info > 0	run time	the call returns `info`; Julia throws `InfoError`; outputs unspecified

The first is an assumption, the second a conditional theorem, the third a measurement. Only the third looks at numbers, and only at the numbers that the chosen kernels happen to factorize.

CHAPTER 4

Matrix Dimensions: Static, Named, Checked at Run Time

Answer. Dimensions are checked at three levels, by three different mechanisms. Static sizes are part of the MLIR tensor type and are checked by op verifiers. Run-time sizes are *named* in the Julia (or Python) frontend, and the frontend checks at trace time that names agree, for example that the inner sizes of a product are the same symbol. In MLIR the names survive only as the argument attribute `ulm_lam.dims`, which an analysis uses to find the size of each dimension, but which no verifier uses to check agreement. When the compiled function is called, the Julia runtime binds each name to one integer and rejects arrays that disagree. The generated C function trusts the sizes it receives.

4.1 Static sizes: checked by the verifier

With `Mat(40, 6, :full_rank)` the sizes become part of the type, `tensor<6x40xf64, ...>`, and every op verifier compares them (Section 2.3): `factor 0 has 4 columns but factor 1 has 5 rows`. This is ordinary type checking with sizes in the types. The test suite compiles a static GLS (`gls_static`) next to the dynamic one; the derivation is the same, and the costs are exact instead of estimated (Section 8.1).

4.2 Run-time sizes: anonymous in MLIR

A dynamic dimension is written `?`. Two `?` are not known to be equal or different, so the verifier accepts any product of two `tensor<?x?xf64>` (`v2` in Section 2.3). This is MLIR's builtin type system, and it is deliberately weak: a `?` is "an `index` value that you can ask for with `tensor.dim`", not a symbol.

4.3 Named sizes: the frontend checks, MLIR records

Input	`Mat(:n, :m)`, `Mat(:n, :n)`, `Vec(:n)`: the same symbol means the same size.
Representation	in Julia, the symbols themselves (`MatType.rows`, `.cols`); in MLIR, `index` parameters `%s.n`, `%s.m` and `ulm_lam.dims = [1, 0]` on each matrix parameter.
Rule	at trace time, products and sums compare symbols; in MLIR, `dimOf` maps each dynamic dimension to an `index` value.
Output	a `LamError` at trace time, or IR in which every `tensor.empty` gets its sizes from the parameters.
Theory	named symbolic dimensions, FG §10.4 *Symbolic and dynamic shapes*; indexed types, FG §1.8 *Dependent and indexed types.*

At trace time. The overloaded `*` compares `ta.cols == tb.rows`, where both may be symbols (Section 1.3). So the frontend rejects

```
@lam bad(A::Mat(:n, :m), B::Mat(:n, :m)) = A * B
```

with `LamError: product of nxm and nxm: m columns but n rows` (`outputs/gls.out`). The check is syntactic equality of names: a product of `Mat(:n, :m)` and `Mat(:m, :k)` has the type `Mat(:n, :k)`, and no arithmetic on sizes is ever needed, because matrix algebra only ever equates dimensions.

In MLIR. The names become positions. The function gets one `index` parameter per distinct symbol, and each matrix parameter says which of them its dynamic dimensions are:

```
func.func @gls(%s.n: index, %s.m: index,
    %p.X: tensor<?x?xf64, ...> {ulm_lam.in, ulm_lam.dims = [1, 0]}, ...
```

The function `dimOf` (`lib/UlmLamDialect.cpp:1269`) answers "which `index` value is dimension i of this matrix?" for any value in the pipeline: for a parameter, from `ulm_lam.dims`; for a product, from its first or last factor; for a transpose, from the other dimension of its operand; for a DPS kernel, from its destination; for a loop result, from its initial value; and so on. Kernel selection uses it to size every new matrix (`tensor.empty(%arg0, %arg1)` in Section 1.7) and to emit the run-time cost formulas of Chapter 8. Without the attribute it gives up (`outputs/dims.out`):

```
v2-mul-dynamic.mlir:2:8: error: ulm-lam-select-kernels: unknown size of a dynamic dimension (missing
ulm_lam.dims?)
```

What MLIR does not check. `ulm_lam.dims` is used to *find* sizes, not to *check* them. In `v11-dims-mismatch.mlir` A is $n \times k$ and B is $n \times k$, so AB needs $k = n$:

```
func.func @f(%n: index, %k: index,
    %a: tensor<?x?xf64> {ulm_lam.in, ulm_lam.dims = [0, 1]},
    %b: tensor<?x?xf64> {ulm_lam.in, ulm_lam.dims = [0, 1]})
    -> (tensor<?x?xf64> {ulm_lam.out, ulm_lam.dims = [0, 1]}) {
  %p = ulm_lam.mul %a, %b : tensor<?x?xf64>, tensor<?x?xf64> -> tensor<?x?xf64>
  return %p : tensor<?x?xf64>
}
```

The verifier accepts it, and kernel selection emits `ulm_lamk.gemm ... ins(%arg2, %arg3 ...) outs(%0 ...)` with `%0 = tensor.empty(%arg0, %arg1)` (`outputs/dims.out`). If this IR were compiled and called with $k \neq n$, the BLAS call would read outside B. The Julia frontend never produces such IR, because it rejects the program at trace time; hand-written IR is not protected.

4.4 At the call: binding names to integers

When the compiled function is called, `bind_sizes!` (`julia/UlmLam/src/runtime.jl:326`) walks the arguments and binds each symbol to the size it finds:

```
for (d, v) in ((t.rows, r), (t.cols, c))
    if d isa Symbol
        get!(env, d, v) == v ||
            throw(LamError("$(what()): size :$d = $v, but $(env[d]) from an earlier argument"))
    elseif d != v
        throw(LamError("$(what()): expected $t, got $r x $c"))
    end
end
```

The first argument that mentions `:n` fixes it; every later occurrence must agree: `LamError: gls: y: size :n = 3, but 4 from an earlier argument`. The sizes are then passed to C as `size_t`. The C function is called only with consistent sizes, but it does not know that: called directly from C with inconsistent sizes, it reads or writes out of bounds.

Theory (FG §1.8, FG §10.4, FG §6.1). Types indexed by size variables, with equality as the only relation, are the simplest case of FG §1.8 *Dependent and indexed types* [7]: `Mat(:n, :m)` is like a type Mat n m indexed by n, m, and the product has type Mat n m → Mat m k → Mat n k. In ulm.lam this discipline lives in the Julia frontend (checked at trace time) and in the Julia runtime (checked at the call); MLIR keeps named dimensions only as data for an analysis. FG §10.4 *Symbolic and dynamic shapes* describes the same split between anonymous dynamic dimensions (MLIR's `?`) and named symbolic ones (here: `:n`), and FG §6.1 *Symbolic compile-time parameters* the general idea of values that are unknown but constrained.

Not the same thing. A shape parameter in a type is not a dependent type system. There is no type-level computation, no proof that sizes agree inside MLIR, and no constraint between sizes other than equality of names. In particular the fact $n \geq m$, which full *column* rank needs, cannot be stated: `fullColumnRank` assumes it for dynamic sizes (Section 1.6), and nothing checks it before `potrf` of S fails or succeeds.

A hypothetical extension. *(Not in ulm.lam.)* Stating size constraints would close that gap:

```
# schematic, not existing ulm.lam syntax
@lam function gls(X::Mat(:n, :m, :full_rank), M::Mat(:n, :n, :spd), y::Vec(:n);
                  require = (:n >= :m))
```

The frontend would add the constraint to the function (an attribute, or an assumption op as in MLIR's `shape` dialect), `fullColumnRank` could use it instead of assuming it, and the runtime would check it with the other sizes. This is the "symbolic shape with constraints" of FG §10.4.

CHAPTER 5

Values, Uses and Effects

Answer. An SSA (static single assignment) value in ulm.lam may be used any number of times; using it twice means reading one matrix twice, not copying it. Rewrites that would duplicate a *computation* check that the value has a single use. Whether an operation may be removed, moved out of a loop or merged with another is decided by MLIR's generic passes from per-op effect declarations: all `ulm_lam` ops are `Pure`, `ulm_lamk` ops have no effects on tensors and read/write effects on memrefs, and a call of an external function may do anything. Moving a pure op out of a loop is sound for the declared facts. If a declaration is false, the move becomes visible: a factorization hoisted out of a loop that runs zero times can still report failure.

5.1 One value, several uses

In Figure 1.2, $W = L_1^{-1}X$ has two uses and L_1 has two uses. Each is computed once and stored once (Section 1.8). Nothing in MLIR restricts how often an SSA value is used, and nothing in ulm.lam needs a restriction.

The restriction appears where a rewrite would copy an *expression* into several places. `Flatten` merges an inner product into the outer one only if the inner product has no other use (`lib/UlmLamDialect.cpp:578`):

```
for (Value v : op->getOperands()) {
    auto inner = v.template getDefiningOp<OpT>();
    if (inner && inner->hasOneUse()) {          // mul(a, mul(b, c)) -> mul(a, b, c)
        llvm::append_range(operands, inner->getOperands());
        changed = true;
    } else {
        operands.push_back(v);
    }
}
```

If $P = AB$ is used in PC and in $P + C$, flattening PC into ABC would compute AB a second time for the sum. With the check, P stays a value of its own. `examples/variants.jl` has exactly this program (`shared`); `@code_lam` shows AB computed once:

```
shared(A::n×k, B::k×n, C::n×n)
  gemm     T1 ← A B
  gemm     T2 ← T1 C
  copy     T3 ← T1
  axpy     T3 ← T3 + C
  return T2, T3
```

(The copy has another reason: the sum is accumulated in place into a new matrix that is returned in Julia's column-major layout, so T_1 is copied there transposed before C is added. It is not a copy for the second use; T_1 is read, not overwritten, by the first.) `PushTranspose`, which turns $(ABC)^\mathsf{T}$ into $C^\mathsf{T}B^\mathsf{T}A^\mathsf{T}$, has the same one-use condition (`lib/UlmLamDialect.cpp:641`).

Theory (FG §1.1, FG §2.7). In the terms of FG §1.1 *Structural rules and substructural type systems*, SSA values obey both *contraction* (a value may be used twice) and *weakening* (a value may be unused): MLIR's value discipline is the ordinary, non-substructural one. That is safe because tensors have value semantics, FG §2.7 *Value semantics vs reference/memory semantics in IRs*: a use can never observe another use. The one-use checks above are not a typing rule; they are a cost side condition on rewrites, so that a rewrite never duplicates work.

Not the same thing. An SSA value with several uses does not imply several copies of its storage. Copies are made by bufferization, and only when a use would *overwrite* storage that another use still reads (Chapter 6). Conversely, linear types (FG §1.2) would restrict the number of uses; nothing in MLIR or ulm.lam does.

5.2 Effects decide what may move

Input	an op inside a loop, or an op whose results are unused.
Representation	the op's `MemoryEffectOpInterface`: the trait `Pure` (no effects, speculatable), or a `getEffects` method.
Rule	generic passes: DCE (dead code elimination) removes ops without effects whose results are unused; LICM (loop-invariant code motion) hoists pure ops whose operands are defined outside the loop; CSE merges identical pure ops.
Output	the program with ops removed, moved or merged.
Theory	FG §1.6 *Effect systems and algebraic effects*; FG §3.2 *Traits and interfaces.*

The declarations.

- Every `ulm_lam` op is `Pure` (the base class `UlmLam_Op`, `include/ulm_lam/UlmLamOps.td:23`). They are mathematical functions of their operands.
- `ulm_lamk` ops implement `getEffects` with `dpsEffects` (`lib/UlmLamDialect.cpp:808`): memref operands are read, DPS destinations are also written; "Tensor operands have value semantics (no effects)." The same op is effect-free on tensors and effectful on memrefs.
- `ulm_lamk.call` calls a function written in Julia or C; it adds a read and a write of the default resource, i.e. of any memory (`lib/UlmLamDialect.cpp:1150`): "the call is never hoisted, duplicated or removed".

Removing. On tensors, a kernel whose results are unused is dead, even a factorization with its `info`: in `v12-dead-potrf.mlir` the canonicalizer erases the tensor `potrf` and keeps the memref one (`outputs/dead.out`):

```
func.func @f(%arg0: tensor<4x4xf64>, %arg1: memref<4x4xf64>) -> tensor<4x4xf64> {
  %info = ulm_lamk.potrf uplo(lower) outs(%arg1 : memref<4x4xf64>)
  return %arg0 : tensor<4x4xf64>
}
```

A run-time check that is a by-product of a computation lives only as long as the computation is needed. On tensors this is correct by construction: `info` is an ordinary result, and an unused result is not observable.

5.3 Moving a factorization out of a loop

The loop in `examples/variants.jl` applies M^{-1} in every step:

```
@lam function iterate(M::Mat(:n, :n, :spd), x0::Vec(:n), steps::Index)
    loop(steps, x0) do i, x
        (inv(M) * x,)
    end
end
```

The traced IR has the inverse inside the loop body; after normalization, inference and `loop-invariant-code-motion` it is outside (`outputs/iterate-traced.mlir`, `outputs/iterate-licm.mlir`):

```
// traced
%v4 = scf.for %v5 = %v2 to %p.steps step %v3 iter_args(%v6 = %p.x0) -> (tensor<?x1xf64>) {
  %v7 = ulm_lam.inverse %v1 : tensor<?x?xf64> -> tensor<?x?xf64>
  %v8 = ulm_lam.mul %v7, %v6 : tensor<?x?xf64>, tensor<?x1xf64> -> tensor<?x1xf64>
  scf.yield %v8 : tensor<?x1xf64>
}
// after LICM
%0 = ulm_lam.inverse %arg1 : ... -> tensor<?x?xf64, #ulm_lam.props<square, symmetric, spd, ...>>
%1 = scf.for %arg4 = %c0 to %arg3 step %c1 iter_args(%arg5 = %arg2) -> (tensor<?x1xf64>) {
  %2 = ulm_lam.mul %0, %arg5 : ...
  scf.yield %2 : tensor<?x1xf64>
}
```

LICM moved `inverse` because its operand is defined outside the loop and the op is `Pure`. Kernel selection then factorizes M once, before the loop; the loop body is two `trsv` (`outputs/iterate-selected.mlir`). Kernel selection would place the factorization there even without LICM, because it factorizes a matrix directly after its definition (Section 1.7); two independent mechanisms agree.

5.4 Speculation and a false declaration

`Pure` in MLIR means `NoMemoryEffect` *and* `AlwaysSpeculatable`: the op may be executed even on paths where the original program would not execute it. For `ulm_lam.inverse` of an SPD matrix this is harmless: the inverse exists, and computing its factorization early changes nothing observable. For an M that is declared SPD but is not, it changes the outcome (`examples/speculate.jl`, `outputs/speculate.out`):

```
steps = 0: InfoError: iterate: info 1: Cholesky factorisation of v2: not positive definite
steps = 1: InfoError: iterate: info 1: Cholesky factorisation of v2: not positive definite
```

With `steps = 0` the formula never applies M^{-1}; its value is x_0. The compiled function factorizes M before the loop and reports the failure anyway. Relative to the declared facts (M is SPD) the transformation is exact; relative to the actual input it is not.

Theory (FG §1.6, FG §3.2). FG §1.6 *Effect systems and algebraic effects* explains why compilers need effect information [8]: a computation without effects may be removed, hoisted or merged. MLIR attaches that information to op classes and instances through traits and interfaces, FG §3.2 *Traits and interfaces*, and LICM, CSE and DCE are written once against them. The example shows the other half of the contract: correctness depends on the declarations being honest *for the inputs that occur*. `ulm_lam.inverse` is pure on the domain where the inverse exists; the domain is guaranteed only by a declaration.

Not the same thing. Failure is not modeled as an effect. In an effect system, "may fail" would be an effect like "may raise", and a may-fail operation would not be speculatable. In ulm.lam the possibility of failure is outside the `ulm_lam` level entirely (the ops are total over the declared domain); it appears at the `ulm_lamk` level as an ordinary result value, `info`, and only `ulm-lam-info` turns it into control flow (Section 5.5).

5.5 Failure as a value, then as control flow

`ulm_lamk.potrf` returns `info` as an `i64` result. After bufferization, `ulm-lam-info` (`lib/UlmLamInfo.cpp:1`) numbers every check of a function and rewrites the rest of the block into an `scf.if`:

```
%info = ulm_lamk.potrf ... outs(%A)
%code = arith.select (%info != 0), k, 0
%s = scf.if (%code == 0) -> i64 { <rest>; yield <status of rest> }
                        else { yield %code }
```

(This listing is the schematic from the pass's header comment.) A loop carries the status, so the iterations after a failure do nothing. The function returns the status; outputs after a failure are unspecified, as in LAPACK. The descriptions of the checks go into the function attribute `ulm_lam.info` and from there into the C++ interface comment and Julia's `InfoError`.

> **Theory.** This is error handling by explicit status passing: every operation that can fail returns a code, and the compiler threads it through the rest of the computation. An exception or an algebraic effect with a handler (FG §1.6) would express the same thing with a non-local exit; `scf.if` nesting is how a structured IR without exceptions encodes it.

5.6 Regions in MLIR and regions in memory management

ulm.lam's IR contains MLIR *regions*: the body of `scf.for`, the branches of `scf.if`, the body of every `func.func`. These are nested blocks of code with lexical scoping of SSA values, FG §2.6 *Regions and structured control flow in the IR*.

> **Not the same thing.** MLIR regions are not the regions of region-based memory management, FG §1.5 *Region types and lifetimes*, where a region is a pool of memory whose lifetime the type system tracks. The closest thing in ulm.lam is the workspace (Section 6.4): one block of memory per call, from which all temporaries are carved and which is released (or kept by the caller) at the end. That is an arena with a lifetime of one call, managed by a pass and a calling convention, not by types.

CHAPTER 6

From Immutable Values to Mutable Storage

Answer. Kernel selection produces BLAS and LAPACK kernels on *tensors*, in destination-passing style: each kernel names the value it overwrites, and returns the new value. MLIR's One-Shot Bufferize then assigns buffers. It lets a kernel overwrite its destination in place when the destination's buffer may be written and no later use reads the old value; otherwise it inserts a copy. Read-only parameters are protected by an attribute, so the caller's M is copied before `potrf` overwrites it, unless the caller declares it overwritable. After bufferization, ulm.lam's own pass places all temporaries in one caller-provided workspace at disjoint offsets. No type system is involved: in-place updates are justified by an analysis, and the absence of aliasing by construction.

6.1 Destination-passing style kernels

Input	a kernel that BLAS executes in place, $B \leftarrow L^{-1}B$.
Representation	`ulm_lamk.trsm ... ins(%L) outs(%B) -> tensor` on tensors; the same op with no result on memrefs.
Rule	the `outs` operand is the DPS "init": on tensors, result i is the new value of init i and has its type (`verifyDps`).
Output	a value-semantic op whose natural buffer is that of its `outs` operand.
Theory	FG §10.3 *Bufferization and destination-passing style.*

All kernels share one ODS base class (`include/ulm_lam/UlmLamKernelOps.td:30`):

```
class UlmLamk_Op<string mnemonic, string init, list<Trait> traits = []>
    : Op<UlmLamk_Dialect, mnemonic, !listconcat(traits, [
        DestinationStyleOpInterface,
        DeclareOpInterfaceMethods<MemoryEffectsOpInterface>,
        DeclareOpInterfaceMethods<UlmLam_PropertyInferenceOpInterface>,
        DeclareOpInterfaceMethods<UlmLamk_KernelCostOpInterface>])> {
  let results = (outs Variadic<2DTensorOf<[F32, F64]>>:$result);
  ...
}
```

The variadic result is the trick that lets one op serve both worlds: on tensors it has one result (the new value), on memrefs none. The GLS `trsm`, before and after bufferization:

```
%3 = ulm_lamk.trsm side(left) uplo(lower) trans(none) diag(non_unit) alpha(%cst_0)
       ins(%result : tensor<?x?xf64>) outs(%2 : tensor<?x?xf64>) -> tensor<?x?xf64>
ulm_lamk.trsm side(left) uplo(lower) trans(none) diag(non_unit) alpha(%cst)
       ins(%view : memref<?x?xf64>) outs(%view_2 : memref<?x?xf64>)
```

The bufferization model for every kernel (`lib/UlmLamDialect.cpp:1168`) derives from MLIR's `DstBufferizableOpInterfaceExternalModel`, which supplies the analysis (which operands are read, which are written, which result aliases which destination). ulm.lam implements only `bufferize()`: create the same op on buffers and replace each tensor result by its destination's buffer. The comment says it in one line: "Analysis (in-place decisions, copies) comes from the DPS default model."

6.2 Who may be overwritten

One-Shot Bufferize treats function arguments as writable unless told otherwise. `ulm-lam-results-to-out-params` tells it (`lib/UlmLamResultsToOutParams.cpp:344`): a parameter marked `ulm_lam.in` (the default) gets `bufferization.writable = false`, one marked `ulm_lam.inout` gets `true`. Results become out parameters that the caller allocates.

Variant	Copies	What changes
`gls`: X, M, y `in`	3 (M, X, y)	`potrf` and `trsv` work on copies in the workspace
`gls_inout`: M, y `inout`	1 (X)	`potrf uplo(lower) outs(%arg1 ...)` works on the caller's M; the caller's M and y are destroyed, as with LAPACK

Both lines are checked by `examples/gls/check.sh` (Appendix A). In Julia, `InOut(Mat(...))` declares the second kind of parameter. The intent is declared by the programmer; that the generated code respects it is analysis (One-Shot Bufferize's conflict analysis honors `writable = false`).

6.3 Loop state in place

A loop-carried matrix should live in one buffer for the whole loop. One-Shot Bufferize can arrange that only if the value yielded at the end of the body is computed in the buffer of the iteration argument. Kernel selection makes sure of it (`lib/UlmLamPasses.cpp:358`): if the yielded value was computed elsewhere, it inserts a `bufferization.materialize_in_destination` into the iteration argument. In `iterate` (`outputs/iterate-selected.mlir`):

```
%0 = scf.for %arg4 = %c0 to %arg3 step %c1 iter_args(%arg5 = %arg2) -> (tensor<?x1xf64>) {
  %1 = ulm_lamk.trsv uplo(lower) trans(none) diag(non_unit) ins(%result : ...) outs(%arg5 : ...) -> ...
  %2 = ulm_lamk.trsv uplo(lower) trans(trans) diag(non_unit) ins(%result : ...) outs(%1 : ...) -> ...
  %3 = bufferization.materialize_in_destination %2 in %arg5 : (...) -> tensor<?x1xf64>
  scf.yield %3 : tensor<?x1xf64>
}
```

Here both solves already write into `%arg5`'s buffer, and the materialization costs nothing. For $x \leftarrow Fx$, which cannot be computed in place by `gemv`, it becomes one copy per iteration instead of one allocation per iteration.

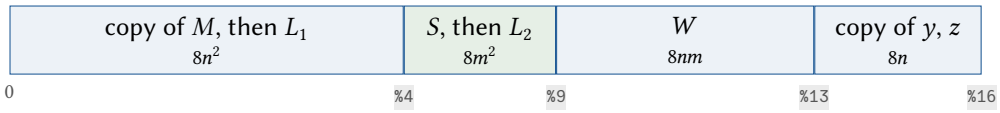
6.4 One workspace for all temporaries

Input	the bufferized function, with a `memref.alloc` per temporary.
Representation	after `ulm-lam-workspace`: two extra parameters, a byte buffer and a size/protocol word; every temporary a `memref.view` into the buffer.
Rule	offsets are the running sum of the temporaries' sizes, each rounded up to 64 bytes; the size is a formula in the run-time sizes.
Output	no allocation inside the function (unless the caller asks for it).
Theory	an arena allocator with a LAPACK-style `lwork` protocol; buffer assignment as storage allocation, FG §10.3 *Bufferization and destination-passing style*, by analogy with register assignment, FG §4.1 *SSA destruction: parallel copies, lost-copy and swap problems.*

The protocol (`lib/UlmLamWorkspace.cpp:1`): with `*workspace_info == -1` the function only stores the required size and returns; with `0` it allocates the block itself and frees it at the end; with a positive value it uses the given block, or returns info $= -i$ if the block is too small. The Julia runtime queries the size

once per combination of run-time sizes and keeps one workspace per task, so repeated calls allocate nothing.

For GLS the bufferized IR computes four aligned sizes and their offsets (the `arith` ops at the top of `6-bufferized.mlir`). Figure 6.1 shows the layout and the numbers for the example of Section 1.10.



$n = 4,\ m = 2$: offsets 0, 128, 192, 256; total 320 bytes.
$n = 2000,\ m = 50$: offsets 0, 32 000 000, 32 020 032, 32 820 032; total 32 836 032 bytes.

Figure 6.1: The GLS workspace: one block, four views at 64-byte aligned offsets (each size rounded up to a multiple of 64). β is not in it: it is written directly into the caller's output.

Every temporary has its own region of the block for the whole call. The pass does not overlap temporaries with disjoint lifetimes (in GLS that would not help: while z is computed, L_1, L_2, W and z are all live). It does let the branches of an `scf.if` share their part, because only one of them runs. Reuse happens *across* calls, through the caller's workspace.

6.5 Aliasing

BLAS and LAPACK assume that an output does not overlap an input (Fortran's rules). ulm.lam guarantees this by construction rather than by analysis:

- temporaries are views at disjoint offsets of one block;
- read-only parameters are never written (`writable = false`);
- a kernel reads its `ins` and overwrites only its `outs`, and bufferization gives an `outs` operand that is still needed elsewhere a copy.

MLIR's generic passes see less. The memref effects of `ulm_lamk` ops are all on the `DefaultResource` (Section 5.2), so any two kernels on memrefs are treated as possibly touching the same memory. That is conservative and therefore sound; it costs nothing here, because all optimization happens before bufferization, on values.

Theory (FG §5.5, FG §2.7). FG §5.5 *Alias and points-to analysis: Andersen, Steensgaard, type-based* answers "may these two pointers refer to the same memory?". ulm.lam avoids the question: on tensors there is no memory and hence no aliasing (FG §2.7), and after bufferization the program is no longer optimized. The one analysis that does reason about aliasing is One-Shot Bufferize's, which tracks which tensor values are equivalent to which buffers in order to decide in-place writes.

Not the same thing. Bufferization and storage reuse do not imply linear typing. A linear type system (FG §1.2) or a uniqueness type (FG §1.4) would *guarantee*, by a typing rule, that a value is not used after it is updated in place. Here a value may be used any number of times, and the bufferizer *decides*, for each update, whether an in-place write is safe; if not, it copies. The program is never rejected for using a value twice; it only gets slower.

6.6 What each lowering keeps and drops

Table 6.1 answers "what is preserved or lost during lowering". The pattern is that of FG §3.3 *Progressive lowering and the dialect conversion framework*: each stage keeps exactly what a later stage still needs.

Information	**ulm_lam**	**ulm_lamk, tensors**	**ulm_lamk, memrefs**	**C++**
matrix properties	encodings	kernel choice, `uplo`, `diag`; encodings until `strip-properties`	gone	routine names (`trlsv`, `trusv`)
transposition	`transpose` ops	`trans` flags	`trans` flags	swapped strides
the algebra	`mul`, `inverse`	kernels	kernels	calls
value semantics	tensors	tensors (DPS)	buffers	pointers
run-time sizes	`?` + `ulm_lam.dims`	`index` values	`index` values	`size_t` arguments
in/out intent	arg. attributes	arg. attributes	copies or none	interface comment
failure	—	`info` result	`info`, then `scf.if`	return value

Table 6.1: Information through the stages. Properties are consumed by kernel selection: after it, the program no longer says *why* a Cholesky factorization was chosen, only *that* it was.

The properties are needed only by kernel selection and are dropped right after it; the DPS ties are needed by bufferization and disappear with it; transposition survives down to the C++ call, as strides.

CHAPTER 7

From Algebra to Kernels

Answer. Three different mechanisms turn the formula into calls. Local algebraic identities ($(A^\mathsf{T})^\mathsf{T} = A$, $(AB)^\mathsf{T} = B^\mathsf{T}A^\mathsf{T}$, flattening) are *rewrite patterns* applied by MLIR's greedy driver. The choice of kernels is a *global optimization* written as an ordinary C++ pass: it needs whole products, a cost model, dominance and a memo table, which a local pattern cannot see. The final translation to C++ is a *dialect conversion* with a type converter, the textbook use of MLIR's conversion framework. All three preserve the value over the real numbers; none of them preserves floating-point results bit for bit, and kernel selection changes the numerical behavior on purpose.

7.1 Normalization: local rewrites to a fixpoint

Section 1.5 showed the patterns at work. The complete set (`lib/UlmLamDialect.cpp:562` and `lib/UlmLamPasses.cpp:74`):

Pattern	Rewrite	Side condition
`Flatten<MulOp>`, `Flatten<AddOp>`	$A(BC) \Rightarrow ABC$, $A + (B + C) \Rightarrow A + B + C$	inner op has one use
`FoldTranspose`	$(A^\mathsf{T})^\mathsf{T} \Rightarrow A$; $A^\mathsf{T} \Rightarrow A$	A symmetric (from its type)
`PushTranspose`	$(ABC)^\mathsf{T} \Rightarrow C^\mathsf{T}B^\mathsf{T}A^\mathsf{T}$, $(A + B)^\mathsf{T} \Rightarrow A^\mathsf{T} + B^\mathsf{T}$, $(\alpha A)^\mathsf{T} \Rightarrow \alpha A^\mathsf{T}$	inner value has one use
`DropIdentity`	$AIB \Rightarrow AB$	—
`FoldInverse`	$(A^{-1})^{-1} \Rightarrow A$	—
`FoldScaleOne`, `FoldScaleScale`	$1A \Rightarrow A$, $a(bA) \Rightarrow (ab)A$	—
`ForwardDropping Assume`	uses of `assume(x)` in algebra ops $\Rightarrow$ uses of x	the assume only drops facts

Every rewrite that may produce a value with *stronger* properties than the one it replaces wraps it in an `assume` back to the old type (`replaceKeepingType`), so that the IR stays valid for the result-type check of Section 2.4; `ForwardDroppingAssume` then lets the algebra see the stronger value again.

Theory (FG §3.4, FG §3.5). These are rewrite rules with side conditions, run by a greedy driver until none applies: FG §3.4 *Pattern rewriting: greedy drivers, canonicalization, PDL*. The side conditions "has one use" make the rules depend on the *graph* (sharing), not only on the term, which is typical of rewriting on SSA. Termination: every rule removes a transpose, inverse, identity or scale node, or moves a transpose toward the leaves, or reduces nesting; FG §3.5 *Term rewriting theory: confluence, termination, strategies* gives the general method (a well-founded measure that every rule decreases).

7.2 Kernel selection is a global optimization

Section 1.7 described what the selector does for GLS. Why is it not a set of patterns, or a dialect conversion?

- **It decides for a whole product at once.** The cheapest evaluation of $X^\mathsf{T} L_1^{-\mathsf{T}} L_1^{-1} X$ depends on all four leaves; a pattern that sees one `mul` at a time and rewrites it greedily cannot run a dynamic program over its parenthesizations.
- **It shares across products.** W is computed for the first product and reused, transposed, by the second; the selector keeps a map from leaf sequences to computed values (`computed`) and checks dominance before it reuses one.
- **It places code elsewhere.** Factorizations go right after the definition of the factorized matrix, which may be outside a loop, not at the product that needs them.
- **It may fail with a diagnostic.** An explicit inverse in a position that no kernel sequence covers is reported ("no kernel sequence for this product"), not miscompiled.

So the pass walks the function, collects products and sums, and emits kernels with an `OpBuilder`. The dynamic program (`lib/UlmLamPasses.cpp:1058`) fills two tables, cost and choice, over intervals $[i, j]$ of leaves. For each split it considers:

Choice	Kernel	Cost (flops)
`Gemm`	`gemm` / `gemv` of two sub-intervals	$2\,m\,k\,n$
`SolveLeft`	`trsm` / `trsv` / `getrs` applied to the right part, in place	$m^2 n$ ($2m^2 n$ with LU), plus mn for a transposed operand
`SolveRight`	`trsm` side right / `getrs` side right	mn^2 ($2mn^2$)
`Gram`	`syrk`: $Z^\mathsf{T} Z$ when the left half mirrors the right half	$n^2 r$
`Reuse`	a sub-product computed by an earlier product	0

Factorization costs are not part of the dynamic program: every inverted matrix is factorized anyway (Cholesky $n^3/3$, LU $2n^3/3$). One decision does weigh them: whether $(AB)^{-1}$ is formed and factorized or expanded into $B^{-1}A^{-1}$ (`invertFactorwise`, `lib/UlmLamPasses.cpp:816`).

> **Theory (FG §4.5, FG §3.3).** Instruction selection covers an expression tree with machine-instruction patterns of known cost; the optimal cover of a tree is found by dynamic programming (Aho, Ganapathi and Tjiang [4]), FG §4.5 *Instruction selection: tree/DAG tiling, SelectionDAG vs GlobalISel.* Here the "instructions" are BLAS and LAPACK routines, the tree is a chain of leaves, and the patterns are `gemm`, the solves and `syrk`. Two features go beyond tree tiling: the Gram pattern matches a *symmetry* of the interval (the left half is the transpose of the right), and `Reuse` is CSE across products, up to transposition, which makes the problem a DAG covering problem. Both follow Linnea [2], whose generalized matrix chain algorithm [3] the dynamic program implements.

7.3 Lowering to C++: a dialect conversion

Input	the bufferized module: `ulm_lamk` on memrefs, `memref`, `scf`, `arith`, `func`.
Representation	a `ConversionTarget` in which only EmitC is legal; a `TypeConverter` from memrefs to pointers.
Rule	one `OpConversionPattern` per kernel and memref op, plus the upstream `arith`/`scf`/`func`-to-EmitC patterns; `applyPartialConversion`.
Output	EmitC, printed as C++ by `mlir-translate --mlir-to-cpp`.
Theory	FG §3.3 *Progressive lowering and the dialect conversion framework*; FG §4.6 *Legalization: types, operations, and ABI lowering.*

The setup (`lib/ConvertUlmLamkToUlmBLAS.cpp:1154`):

```
TypeConverter tc;
tc.addConversion([](Type t) -> std::optional<Type> {
    if (emitc::isSupportedEmitCType(t)) return t;
    return std::nullopt;
});
tc.addConversion([](MemRefType t) -> std::optional<Type> {
    if (!isCArray(t)) return std::nullopt;
    return emitc::PointerType::get(t.getElementType());
});
...
patterns.add<AllocLowering, ..., GemmLowering, ..., TrsmLowering, ...,
             PotrfLowering, GetrfLowering, GetrsLowering, CallLowering>(tc, ctx);
populateArithToEmitCPatterns(tc, patterns);
populateSCFToEmitCConversionPatterns(patterns, tc);
populateFuncToEmitCPatterns(tc, patterns, /*lowerToCpp=*/true);

ConversionTarget conv(*ctx);
conv.addLegalDialect<emitc::EmitCDialect>();
conv.addIllegalDialect<memref::MemRefDialect, lk::UlmLamkDialect,
                       ulm_lam::UlmLamDialect, func::FuncDialect,
                       scf::SCFDialect, arith::ArithDialect>();
if (failed(applyPartialConversion(module, conv, std::move(patterns))))
    return signalPassFailure();
```

This is the case for which the conversion framework is made: a local, op-by-op translation in which types change (a `memref<?x?xf64>` becomes a `double *`), and every op of the source dialects must be gone at the end. The type change is the ABI decision: a matrix is a pointer to row-major data; transposition is in the strides; sizes are `size_t`. A `trsm` with $\alpha = 1$ becomes LAPACK's `trtrs`, which is why `7-gls.cpp` calls `trtrs` for $W = L_1^{-1}X$.

7.4 Algebra over the reals and floating point

Every rewrite in this chapter is an identity of real matrices. None is an identity of floating-point computations, and the compiler does not claim otherwise. Three levels of difference:

Rounding. Reassociating $(AB)C$ into $A(BC)$, replacing $M^{-1}b$ by two triangular solves, or $X^{\mathsf{T}}M^{-1}X$ by $W^{\mathsf{T}}W$, changes the rounding errors. In Section 1.10 the result differs from Julia's in the last bit. Neither is "the" floating-point value of the formula: the formula does not prescribe an evaluation order.

Special values. Floating-point arithmetic is not a field. Kernel selection is careful where that matters: to accumulate into a new destination it copies instead of scaling by $\beta = 0$, because the new memory may hold a NaN (not a number), and the source comment notes that "scaling it by beta = 0 would read uninitialised memory, 0 * NaN = NaN" (`lib/UlmLamPasses.cpp:1262`). Over $\mathbb{R}$, $0 \cdot C + AB = AB$ for every

C; in IEEE (Institute of Electrical and Electronics Engineers) 754 arithmetic only if C contains no NaN or infinity.

Stability. The derivation $S = W^{\mathsf{T}}W$ is the method of normal equations in disguise: it squares the condition number. The facts that the compiler inferred are still true (S is SPD over $\mathbb{R}$), but the floating-point S may be so inaccurate that its Cholesky factorization fails. `examples/conditioning.jl` makes X nearly rank deficient, $X = [\mathbf{1},\ \mathbf{1} + \varepsilon(0,1,2,3)^{\mathsf{T}}]$, keeps M and y of Section 1.10, and compares ulm.lam with a QR (orthogonal triangular) factorization of $L_1^{-1}X$, both against a 256-bit reference (`outputs/conditioning.out`):

ε	$\kappa(L_1^{-1}X)$	**ulm.lam, relative error**	**QR, relative error**
10^{-2}	$1.48 \cdot 10^{2}$	$2.66 \cdot 10^{-13}$	$2.54 \cdot 10^{-15}$
10^{-5}	$1.46 \cdot 10^{5}$	$1.45 \cdot 10^{-6}$	$1.13 \cdot 10^{-12}$
10^{-7}	$1.46 \cdot 10^{7}$	$2.49 \cdot 10^{-2}$	$4.86 \cdot 10^{-10}$
10^{-9}	$1.46 \cdot 10^{9}$	`InfoError: ... info 2: Cholesky factorisation of a computed matrix: not positive definite`	$4.74 \cdot 10^{-8}$

The error of ulm.lam grows like $\kappa^2 u$ ($u \approx 1.1 \cdot 10^{-16}$), that of QR like κu, as the theory of least squares predicts [11]. At $\varepsilon = 10^{-9}$, X still has full column rank and S is SPD, but the computed S is not, and the run-time check fires. The README lists QR among the open points of kernel selection, and notes that a factorization with info $= 0$ can still be useless.

> **Not the same thing.** Algebraic correctness over $\mathbb{R}$ is not floating-point equivalence. Mathematical equivalence alone does not establish identical rounding, numerical stability, or unrestricted legality of reassociation. ulm.lam's rewrites are justified over $\mathbb{R}$; their floating-point quality is a property of the chosen algorithm (Cholesky on normal equations here), which the cost model does not consider: it counts flops, not errors.

CHAPTER 8

Choosing an Evaluation Order

Answer. ulm.lam's cost model counts floating-point operations (flops). With static sizes the dynamic program of kernel selection finds one cheapest plan at compile time. With run-time sizes it cannot, so it runs the dynamic program for several sample sizes, keeps every plan that wins for some sample, and emits all of them behind an `scf.if` that compares their flop counts, computed from the actual sizes when the function is called. The comparison is exact for the plans it has; whether those plans include the best one for every size is a heuristic, not a proof.

8.1 The cost model

Costs are the flop counts in the table of Section 7.2: $2mkn$ for a product, m^2n for a triangular solve, n^2r for a Gram product. They are computed inline by the dynamic program; the ops' `KernelCostOpInterface` methods (`estimateFlops`, `estimateBytes`) exist but are not called (Section 2.6). A dynamic dimension that has no sample value counts as 1000 (`kNominal`, `lib/UlmLamPasses.cpp:190`).

Flops are a crude model: `gemv` moves as many bytes as it computes flops, `gemm` reuses each byte many times, and `potrf` runs at a lower rate than `gemm` for small matrices. ulm.lam contains one measured correction: the choice between FPF^{T} by `gemm` plus `gemmtr` and by Cholesky of P, `trmm` and `syrk` uses a constant $n_0 = 2400$ fitted to OpenBLAS on an Apple M5 (`choleskySandwich`, `lib/UlmLamPasses.cpp:1592`). The README lists a measured cost model, in the spirit of ATLAS (Automatically Tuned Linear Algebra Software), as open work.

8.2 Several plans and a run-time choice

Input	$QK^{\mathsf{T}}V$ with $Q, K \in \mathbb{R}^{n\times d}$, $V \in \mathbb{R}^{n\times e}$, all sizes known only at run time (`test/select/runtime_order.mlir`).
Representation	one n-ary `ulm_lam.mul`; sizes as `index` parameters via `ulm_lam.dims`.
Rule	run the dynamic program for every assignment of $\{10, 1000\}$ to the (at most four) distinct sizes; collect the distinct optimal plans; emit their costs as `index` arithmetic and select with `scf.if`.
Output	two plans, chosen at run time (below).
Theory	symbolic parameters, FG §6.1 *Symbolic compile-time parameters*; multi-versioning as a residual of partial evaluation, FG §8.5 *Partial evaluation and the Futamura projections*; cost models, FG §10.5 *Auto-tuning and learned cost models*.

The output after selection and `canonicalize` (`outputs/runtime_order.mlir`; `%arg0` $= n$, `%arg1` $= d$, `%arg2` $= e$):

```
%0 = arith.muli %arg0, %arg1 : index      // n d
%1 = arith.muli %0, %arg0 : index         // n d n
%2 = arith.muli %1, %c2 : index           // 2 n d n        : Q K^T
%3 = arith.muli %arg0, %arg0 : index
%4 = arith.muli %3, %arg2 : index
%5 = arith.muli %4, %c2 : index           // 2 n n e        : (Q K^T) V
```

```
%6 = arith.addi %2, %5 : index            // cost of plan 0
...                                       // 2 d n e + 2 n d e: cost of plan 1
%13 = arith.addi %9, %12 : index
%14 = tensor.empty(%arg0, %arg2) : tensor<?x?xf64>
%15 = arith.cmpi ule, %6, %13 : index
%16 = scf.if %15 -> (tensor<?x?xf64>) {
  %17 = tensor.empty(%arg0, %arg0) : tensor<?x?xf64>
  %18 = ulm_lamk.gemm trans_a(none) trans_b(trans) ... ins(%arg3, %arg4 ...) outs(%17 ...)   // Q K^T
  %19 = ulm_lamk.gemm trans_a(none) trans_b(none) ... ins(%18, %arg5 ...) outs(%14 ...)    // (Q K^T) V
  scf.yield %19 : tensor<?x?xf64>
} else {
  %17 = tensor.empty(%arg1, %arg2) : tensor<?x?xf64>
  %18 = ulm_lamk.gemm trans_a(trans) trans_b(none) ... ins(%arg4, %arg5 ...) outs(%17 ...)  // K^T V
  %19 = ulm_lamk.gemm trans_a(none) trans_b(none) ... ins(%arg3, %18 ...) outs(%14 ...)    // Q (K^T V)
  scf.yield %19 : tensor<?x?xf64>
}
```

Both branches write into the same destination `%14`, so that bufferization gives the `scf.if` result one buffer instead of one per branch (`lib/UlmLamPasses.cpp:1847`). The README measures the effect: for $n = 4096$, $d = 64$, the formula as written in Julia, $(QK^{\mathsf{T}})V$, takes 43 times as long as ulm.lam, which picks $Q(K^{\mathsf{T}}V)$.

How the plans are found. `plansOf` (`lib/UlmLamPasses.cpp:1495`) samples the corners of the size space, $\{10, 1000\}^s$ for $s \le 4$ distinct sizes, and keeps each distinct optimal plan. For the chain above two plans suffice for all sizes, but in general a plan that is optimal only in the interior of the size space would be missed, and with more than four distinct sizes only the nominal plan is generated.

> **Theory (FG §6.1, FG Chapter 6).** The costs are polynomials in symbolic sizes. Deciding at compile time which plan is cheapest for all admissible sizes is a question about the sign of a polynomial difference over a region: the kind of symbolic reasoning of FG Chapter 6 *Constraints and Symbolic Reasoning*, with nonlinear (polynomial) constraints that Presburger arithmetic does not cover. ulm.lam sidesteps the decision: it samples to find candidate plans and defers the comparison to run time, when the sizes are numbers. The values are unknown at compile time but become known before the computation starts, which is the binding time situation of FG §6.1 *Symbolic compile-time parameters*.

> **Theory (FG §8.5, FG §9.2, FG §10.5).** A function with several specialized versions and a dispatch on run-time values is a residual program in the sense of FG §8.5 *Partial evaluation and the Futamura projections*: everything that does not depend on the sizes was decided at compile time. Unlike a JIT (just-in-time) compiler with speculation, FG §9.2 *JIT compilation: tiering, OSR, deoptimization* (OSR: on-stack replacement), nothing is guessed and nothing needs to be undone; every version is correct for every size. And the cost model is an analytical one, not learned or measured, FG §10.5 *Auto-tuning and learned cost models*.

8.3 Decisions that depend on a trip count

In a loop, $b_i = S^{-1}H^{\mathsf{T}}R^{-1}y_i$ can be evaluated as written in every step, or the operator $P = S^{-1}H^{\mathsf{T}}R^{-1}$ can be formed once before the loop and applied with one `gemv` per step. The second pays off if the loop runs often enough:

$$\text{cost}(P) + \text{trips} \cdot 2\,\text{rows}(P)\,\text{cols}(P) \;<\; \text{trips} \cdot \text{cost}(\text{step}).$$

The trip count is usually known only at run time, so `hoistOperator` (`lib/UlmLamPasses.cpp:1909`) emits the condition and, if it holds, P before the loop, and both variants in the loop under the same condition.

Matrix-vector flops are weighted ten times matrix-matrix flops (`kLevel2 = 10`) in this decision, a second crude correction of the flop model for memory-bound kernels. The Kalman-filter examples (`examples/kalman/`) exercise it.

8.4 Specialization by element type, not by size

One compiled library serves all sizes. What the Julia frontend does specialize is the element type: a call with `Float32` arrays compiles a second library from the same traced text with `f64` replaced by `f32` (`julia/UlmLam/src/runtime.jl:262`). Libraries are cached under a hash of the program text, the compiler's modification time, the backend and the flags.

> **Theory (FG §8.7).** Compiling one copy per element type and choosing it at the call by the argument types is monomorphization, FG §8.7 *Monomorphization vs dictionary passing*. Sizes are handled the other way: one generic code with run-time size parameters, plus the run-time plan selection of Section 8.2 where the order depends on them.

CHAPTER 9

Summary and Navigation

This chapter answers the six questions the companion set out to answer, each with pointers to the sections that show it, and ends with the navigation table from ulm.lam examples to the Field Guide.

9.1 Six answers

1. Which guarantees do ulm.lam's types express? Static sizes (checked, Section 4.1); the element type and rank (checked); a conjunction of up to fifteen matrix facts in the tensor encoding, closed under implication, with no contradictions and consistent with a static shape (checked, Section 2.2); and, inside a function, that every fact on a result follows from the facts on the operands by ulm.lam's rules (checked, Section 2.4). The facts themselves are guarantees only relative to the declarations at the parameters and in `assume`.

2. Which facts are assumed, verified, inferred or checked at run time? Assumed: the properties of parameters and of `assume`, and full *column* rank for dynamic sizes (Section 3.2, Section 1.6). Verified: shapes, contradictions, result types (Chapter 2). Inferred: result properties (Section 3.3). Established by analysis: sizes of dynamic dimensions, reuse under dominance, in-place writes (Section 3.4). Checked at run time: array sizes (Julia/Python runtime), positive definiteness (`potrf`), exact singularity (`getrf`), workspace size (Section 3.5). Not established: symmetry of inputs, $n \geq m$, zeros in the unreferenced triangle, conditioning.

3. What does MLIR provide, and what does ulm.lam implement? MLIR: the tensor and memref types with an encoding slot, ODS, verifier hooks, `InferTypeOpInterface`, the effect interfaces and generic passes (CSE, DCE, LICM, canonicalization), pattern rewriting, dialect conversion, One-Shot Bufferize, EmitC. ulm.lam: the property attribute and its meaning, the verifiers, the inference rules, the effect declarations, the normal form, kernel selection with its cost model, the DPS kernel ops and their bufferization models, the workspace and `info` passes, the lowering patterns, and the frontends (Table 2.1).

4. How do those facts enable transformations? `symmetric` removes transposes (Section 1.5). `spd` selects Cholesky instead of LU, and with it `syrk` instead of `gemm` and two triangular solves instead of an LU solve (Table 3.2). `full_rank` turns $X^{\mathsf{T}}M^{-1}X$ from SPSD into SPD and so decides the factorization of S. Triangularity removes a factorization. Purity lets LICM hoist an inverse out of a loop (Section 5.3). Read-only intents decide copies (Section 6.2).

5. How do value semantics, effects, aliasing and bufferization interact? All optimization happens on tensors, where values have no identity and no aliasing, and effects are absent: uses can be shared, pure ops moved and dead ones removed, including a dead factorization with its check (Chapter 5). DPS kernels name the value they overwrite; One-Shot Bufferize turns each into an in-place write or a copy, honoring read-only parameters (Chapter 6). After bufferization the kernels have read/write effects on one default resource, no further optimization needs alias information, and the workspace pass places temporaries at disjoint offsets. Failure travels as a value (`info`) until it becomes control flow (Section 5.5).

6. Where is the corresponding theory? In the table below.

9.2 Navigation table

From Julia to typed IR

ulm.lam example	MLIR mechanism	Theoretical concept	Field Guide
`@lam` traced into `ulm_lam` ops (Section 1.3)	textual IR from overloaded operators; `func.func`	tracing front end of an embedded DSL	FG §3.7
Julia `if` and loops disappear at trace time (Section 1.3)	—	binding time, staging	FG §8.6

Facts: refinement, inference, assumptions, checks

ulm.lam example	MLIR mechanism	Theoretical concept	Field Guide
`tensor<?x?xf64, #ulm_lam.props<spd>>` (Section 1.4)	attribute in the builtin tensor encoding; `VerifiableTensorEncoding`	refinement type	FG §1.7
`spd` on a 3×4 tensor rejected (Section 2.2)	`verifyEncoding`	well-formedness of refinements	FG §1.7
result declared `spd` rejected without `full_rank` (Section 2.4)	`InferTypeOpInterface`, `isCompatibleReturnTypes`	subtyping by implication; checking mode	FG §1.7, FG §1.10
$X^{\mathsf{T}} M^{-1} X$ inferred SPD (Section 1.6)	`inferReturnTypes`, pass `ulm-lam-infer-properties`	type synthesis	FG §1.10
property sets, closure, "absence = unknown" (Section 3.1)	bit-set attribute, `closure`	lattice, abstract domain, transfer functions	FG §5.1, FG §5.2
loop-carried facts re-declared with `assume` (Section 3.1)	block arguments keep their types	fixpoint over loops (not implemented)	FG §5.1
`assume(S, :spd)` (Section 3.2)	`ulm_lam.assume`, no inference hook	trusted cast into a refinement	FG §1.7
`potrf` returns `info` (Section 5.5)	result value, then `scf.if` from `ulm-lam-info`	run-time check; status passing vs exceptions	FG §1.6

Dimensions

ulm.lam example	MLIR mechanism	Theoretical concept	Field Guide
inner sizes of a product (Section 4.1)	op verifiers on static shapes	sizes in types	FG §1.8
`Mat(:n, :m)` and `ulm_lam.dims` (Section 4.3)	`index` parameters, argument attribute, `dimOf`	named symbolic dimensions; indexed types (by analogy)	FG §10.4, FG §1.8, FG §6.1
sizes bound at the call (Section 4.4)	— (Julia runtime)	run-time shape check	FG §10.4

Values, uses and effects

ulm.lam example	MLIR mechanism	Theoretical concept	Field Guide
W used twice, stored once (Section 5.1)	SSA use lists; `hasOneUse` in patterns	contraction and weakening; value semantics	FG §1.1, FG §2.7
`inverse` hoisted out of a loop (Section 5.3)	`Pure`, LICM	effects decide code motion	FG §1.6, FG §3.2
factorization fails in a zero-trip loop (Section 5.4)	`AlwaysSpeculatable` (part of `Pure`)	speculation; honesty of effect declarations	FG §1.6, FG §3.2
unused tensor `potrf` erased (Section 5.2)	`MemoryEffectOpInterface`, DCE	effect-free ops are removable	FG §1.6
`ulm_lamk.call` never moved (Section 5.2)	read/write of the default resource	worst-case effects of unknown calls	FG §1.6
`scf.for`, `scf.if` bodies (Section 5.6)	MLIR regions	structured control flow (not region memory)	FG §2.6, FG §1.5

Storage

ulm.lam example	MLIR mechanism	Theoretical concept	Field Guide
`trsm ... outs(%B)` on tensors and memrefs (Section 6.1)	`DestinationStyleOp Interface`, `DstBufferizable OpInterfaceExternalModel`	destination-passing style	FG §10.3
copies of M, X, y; none with `InOut` (Section 6.2)	One-Shot Bufferize, `bufferization.writable`	in-place condition; uniqueness (by analogy)	FG §10.3, FG §1.4
loop state in one buffer (Section 6.3)	`materialize_in_ destination`	bufferization of loop-carried values	FG §10.3
one workspace, 64-byte aligned views (Section 6.4)	pass `ulm-lam-workspace`, `memref.view`	arena allocation; buffer assignment	FG §10.3, FG §4.1
no overlap between kernel operands (Section 6.5)	disjoint views; effects on `DefaultResource`	alias analysis (avoided)	FG §5.5, FG §2.7
properties dropped after selection (Section 6.6)	`ulm-lam-strip-properties`	progressive lowering	FG §3.3

Rewriting, kernels and floating point

ulm.lam example	MLIR mechanism	Theoretical concept	Field Guide
normal form (Section 7.1)	`OpRewritePattern`, greedy driver	rewriting to a fixpoint; termination	FG §3.4, FG §3.5
$X^\mathsf{T} M^{-1} X$ as `trsm` + `syrk` (Section 1.7)	custom pass, dynamic program	instruction selection by tiling; matrix chain	FG §4.5
W reused transposed (Section 7.2)	memo table, dominance	CSE up to transposition; DAG covering	FG §4.5
C++ from memrefs (Section 7.3)	`ConversionTarget`, `TypeConverter`, `applyPartialConversion`	dialect conversion; legalization, ABI	FG §3.3, FG §4.6
last-bit differences; NaN-safe accumulation (Section 7.4)	—	real vs floating-point semantics	—
Cholesky fails for nearly rank-deficient X (Section 7.4)	—	conditioning of normal equations	—

Evaluation order and specialization

ulm.lam example	MLIR mechanism	Theoretical concept	Field Guide
flop counts as costs (Section 8.1)	inline formulas; `Kernel CostOpInterface` unused	analytical cost model	FG §10.5
$(QK^\mathsf{T})V$ or $Q(K^\mathsf{T}V)$ at run time (Section 8.2)	`scf.if` on `index` cost polynomials	symbolic parameters; residual program	FG §6.1, FG §8.5
operator hoisted if the trip count pays (Section 8.3)	`scf.if` before and in the loop	run-time decision on a symbolic parameter	FG §6.1
`Float32` library (Section 8.4)	same IR text with `f32`	monomorphization	FG §8.7

The entries marked — in the last column have no entry of their own in the Field Guide; floating-point behavior is covered by the numerical literature [11].

APPENDIX A

Reproducing the Examples

Everything in this companion that is shown as compiler or program output was produced by one script, `examples/run.sh`, at commit `4185542` (`41855425a8b9ea4ee279fd2ea4b31b45b53e1983`) of ulm.lam. Its outputs are kept in `outputs/` next to this document.

A.1 Commands

The companion was added to the repository after commit `4185542`, and later commits change the compiler. To run the examples against the compiler this text describes, check out that commit separately and point the script at it (from the root of the repository):

```
git worktree add ../ulm-lam-4185542 4185542
make -C ../ulm-lam-4185542 plugin tool check   # dialects, passes, ulm-lam-opt; lit tests
julia --project=../ulm-lam-4185542/julia/UlmLam -e 'using Pkg; Pkg.instantiate()'
ULM_LAM_ROOT=$PWD/../ulm-lam-4185542 doc/field-guide-companion/examples/run.sh
cd doc/field-guide-companion && latexmk        # this document (LuaLaTeX)
```

Without `ULM_LAM_ROOT` the script uses the compiler of the current checkout; `outputs/versions.out` records which commit it ran on.

`run.sh` runs:

- `examples/gls.jl`: the GLS example of Chapter 1 in Julia, with the traced IR, exact and computed β, `@code_lam`, and the failing inputs (`outputs/gls.out`); then `ulm-lam-opt` for the intermediate stages that the compile script does not keep (`outputs/gls/1-traced.mlir` to `7-gls.cpp`);
- `examples/variants.jl`: dropped and assumed properties, the loop `iterate`, the shared product, the non-symmetric M (`outputs/variants.out`, `outputs/iterate-*.mlir`);
- `examples/speculate.jl`: the zero-trip loop (`outputs/speculate.out`);
- `examples/conditioning.jl`: nearly rank-deficient X (`outputs/conditioning.out`);
- `ulm-lam-opt` on `examples/verify/v*.mlir`: accepted and rejected IR (`outputs/verify.out`, `dims.out`, `dead.out`);
- `ulm-lam-opt` on `test/select/runtime_order.mlir` (`outputs/runtime_order.mlir`).

A.2 Versions

From `outputs/versions.out`. LLVM is the compiler project that provides MLIR and Clang (its name was once short for Low Level Virtual Machine); `ILP64` marks Julia's BLAS interface with 64-bit integers, longs and pointers (ILP64: integer, long, pointer, 64 bits).

```
ulm-lam commit: 41855425a8b9ea4ee279fd2ea4b31b45b53e1983
Homebrew LLVM version 23.1.4
julia version 1.13.3
BLAS: LBTConfig([ILP64] libopenblas64_p-r0.3.34.dylib)
Homebrew clang version 23.1.4
Python 3.15.2
ProductName: macOS  ProductVersion: 26.6  (arm64)
```

The document was built with LuaHBTeX 1.24.0 (TeX Live 2026) and latexmk 4.87.

A.3 The project's own checks

At the same commit and with the same tools, the project's test suites pass: `make check` (10 lit/FileCheck tests of the passes), `examples/gls/check.sh` (10 checks of the GLS derivation and its bufferization, including the `InOut` variant), `make stages` (6 checks), the Julia tests (23 tests for each of the three backends, 139 vector-space expressions, 19 robustness and 11 Float32 tests, 6 frontend and `@code_lam` tests) and the Python tests (47).

A.4 Files of this companion

`ulm-lam-companion.tex`	main file; chapters in `ch*.tex`, `app-repro.tex`, `references.tex`
`preamble.tex`, `companion.lua`	layout; inline code
`latexmkrc`	LuaLaTeX build
`examples/`	the scripts and IR files above
`outputs/`	their recorded outputs

The Field Guide links point to `modern-compiler-concepts.pdf` in the directory of this PDF; copy the Field Guide there to make them work.

References

Links of the form `doi:` are DOIs (Digital Object Identifiers).

The two documents this companion connects.

- *Modern Compiler Concepts: A Field Guide* (`modern-compiler-concepts.pdf`). Cited as FG with section numbers; links open its named destinations.
- ulm.lam, github.com/michael-lehn/ulm-lam, at commit `4185542`; its design document `doc/ulm-lam.pdf` and README `README.md`.

MLIR documentation. (mlir.llvm.org, as of November 2026)

- Language Reference: values, regions, the builtin tensor type and its encoding.
- Operation Definition Specification (ODS) and Defining Dialect Attributes and Types.
- Interfaces and Traits.
- Side Effects & Speculation: `Pure`, `AlwaysSpeculatable`, and what passes may assume.
- Pattern Rewriting and Canonicalization.
- Dialect Conversion.
- Bufferization: One-Shot Bufferize and destination-passing style.
- EmitC dialect.

[1] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, O. Zinenko. *MLIR: Scaling Compiler Infrastructure for Domain Specific Computation*. CGO (International Symposium on Code Generation and Optimization) 2021. doi:10.1109/CGO51591.2021.9370308

[2] H. Barthels, C. Psarras, P. Bientinesi. *Linnea: Automatic Generation of Efficient Linear Algebra Programs*. ACM (Association for Computing Machinery) Transactions on Mathematical Software 47(3), 2021. doi:10.1145/3446632

[3] H. Barthels, M. Copik, P. Bientinesi. *The Generalized Matrix Chain Algorithm*. CGO 2018. doi:10.1145/3168804

[4] A. V. Aho, M. Ganapathi, S. W. K. Tjiang. *Code Generation Using Tree Matching and Dynamic Programming*. ACM TOPLAS (Transactions on Programming Languages and Systems) 11(4), 1989. doi:10.1145/69558.75700

[5] T. Freeman, F. Pfenning. *Refinement Types for ML* (ML: Meta Language). PLDI (Conference on Programming Language Design and Implementation) 1991. doi:10.1145/113445.113468

[6] P. M. Rondon, M. Kawaguchi, R. Jhala. *Liquid Types*. PLDI 2008. doi:10.1145/1375581.1375602

[7] H. Xi, F. Pfenning. *Eliminating Array Bound Checking Through Dependent Types*. PLDI 1998. doi:10.1145/277650.277732

[8] J. M. Lucassen, D. K. Gifford. *Polymorphic Effect Systems*. POPL (Symposium on Principles of Programming Languages) 1988. doi:10.1145/73560.73564

[9] P. Cousot, R. Cousot. *Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints*. POPL 1977. doi:10.1145/512950.512973

[10] A. Shaikhha, A. Fitzgibbon, S. Peyton Jones, D. Vytiniotis. *Destination-Passing Style for Efficient Memory Management*. FHPC (Workshop on Functional High-Performance Computing) 2017. doi:10.1145/3122948.3122949

[11] N. J. Higham. *Accuracy and Stability of Numerical Algorithms*, 2nd ed. SIAM (Society for Industrial and Applied Mathematics), 2002. doi:10.1137/1.9780898718027 (Chapter 20: the least squares problem, normal equations versus QR).

[12] E. Anderson et al. *LAPACK Users' Guide*, 3rd ed. SIAM, 1999. doi:10.1137/1.9780898719604

[13] J. J. Dongarra, J. Du Croz, S. Hammarling, I. S. Duff. *A Set of Level 3 Basic Linear Algebra Subprograms*. ACM Transactions on Mathematical Software 16(1), 1990. doi:10.1145/77626.79170